

ADKEX
(Authenticated Ding Key Exchange)

June 30, 2026

Submitted To
Institute of Commercial Cryptography Standards

Contents

1	Introduction	4
1.1	Background and Motivation	4
1.2	Contributions	5
1.3	Design Objectives	6
2	Algorithm Description	7
2.1	Preliminaries	7
2.1.1	Ring, Modules, and Notation	7
2.1.2	Noise Distributions	10
2.1.3	Public Parameters	11
2.1.4	Matrix Generation and Sampling Algorithms	13
2.2	Core Functions	14
2.2.1	Reconciliation Mechanism	14
2.2.2	Number-Theoretic Transform (NTT)	16
2.2.3	Compression and Decompression Functions	18
2.3	Specification of DKE	18
2.4	Specification of CCA-Secure DKEM	18
2.5	Specification of ADKEX	21
2.6	Parameter Sets	26
3	Design Rationale	26
3.1	The mathematical hard problem	26
3.2	Criteria for Parameter Selection	26
3.3	Failure probability analysis.	26
3.4	Other considerations	27
4	Performance Evaluation	27
5	Security Statements and Analyses	30
5.1	Theoretical security	30
5.1.1	Security Model	30
5.1.2	The wPFS security for ADKEX.	32
5.2	Analysis with respect to known attacks	36
5.2.1	Attacks against the underlying MLWE problem	36
5.2.2	Refined gate-count estimate	39
5.2.3	About quantum security	41
5.2.4	Attacks against symmetric primitives	41
5.2.5	Attacks exploiting reconciliation failures	42

5.3 Other security properties	43
6 Feature Statements	43

1 Introduction

1.1 Background and Motivation

The development of quantum computing has posed a fundamental challenge to modern public-key cryptography. Widely deployed mechanisms such as RSA and Diffie-Hellman rely on the integer factorization or discrete-logarithm assumptions, both of which can be solved efficiently by a sufficiently large quantum computer using Shor’s algorithm [1]. This has driven the development of post-quantum cryptographic methods that are designed to remain secure against quantum adversaries.

Lattice-based cryptography is one of the most mature directions in post-quantum cryptography. In particular, the Learning With Errors (LWE) problem and its structured variants [2, 3, 4] provide strong foundations for encryption, key establishment, and digital signatures. Lattice-based key establishment relies on noisy algebraic computations: two parties generate noisy values, which are then reconciled to produce matching bit strings.

Ding Key Exchange is an early and important example of this reconciliation-based approach [5, 6]. Its central idea is to use LWE-type noisy shared values together with public helper information, enabling two parties to recover a common key with high probability. In this setting, noise is not removed from the construction; rather, the protocol is designed so that the noise remains sufficiently small for reconciliation to succeed while still supporting security under LWE hardness assumptions.

Modern post-quantum standardization, however, has largely moved toward module-lattice constructions. The scheme formerly known as Kyber [7] has been standardized by NIST as ML-KEM in FIPS 203 [8], while the scheme formerly known as Dilithium has been standardized as ML-DSA in FIPS 204 [9]. These standards show that Module-LWE-based cryptography offers an attractive balance between security, compactness, and efficient implementation.

This motivates the study of Ding-type reconciliation mechanisms in a modern module-lattice setting. Instead of working over vector spaces such as $\mathbb{Z}_q^n$, one may work over modules R_q^k . The public LWE matrix can be replaced by a structured Module-LWE matrix $\mathbf{A} \in R_q^{k \times k}$, generated from a short public seed in the same spirit as ML-KEM. Such a design preserves the reconciliation-based intuition of Ding Key Exchange while making the construction closer to the current module-lattice ecosystem.

Authenticated Key Exchange (AKE) requires more than passive key establishment. In a real network, an adversary may modify, replay, or inject messages. KEM-based AKE protocols [10, 11, 12], effectively enhance security by integrating long-term public-key authentication with fresh ephemeral key material. In the unilateral setting considered here, the responder B owns a long-term KEM key pair, while the initiator A generates an ephemeral KEM key pair for the session. The resulting two-pass protocol provides responder authentication and establishes a fresh session key derived from both long-term and ephemeral KEM contributions.

1.2 Contributions

This proposal presents an improved version of the Ding Key Exchange, referred to as DKE, in the module-lattice setting. The construction adapts the reconciliation-based idea of Ding Key Exchange to modules R_q^k , replacing the vector-space LWE setting with a Module-LWE framework. The arithmetic is aligned with the design philosophy of ML-KEM, and the reconciliation mechanism is refined to match the noise bounds, modulus choices, and correctness requirements of the module-lattice setting.

We construct a two-pass unilateral KEM-based AKE protocol from **DKEM**, a key encapsulation mechanism derived from DKE; this proposal is also submitted to NGCC in parallel. The responder B has a certified long-term KEM public key pk_B and secret key sk_B . The initiator A generates an ephemeral KEM key pair (M_1, sk_e) for the session. At a high level, A sends its ephemeral public key and a DKEM ciphertext associated with B 's certified long-term public key. The responder decapsulates using sk_B , encapsulates via A 's ephemeral public key, and returns the resulting ciphertext. Both parties then combine the corresponding DKEM shared secrets through a transcript-bound KDF.

The final shared secret ss is derived from both the ephemeral shared secret ss_e and the static shared secret ss_s as

$$ss \leftarrow \text{KDF}(ss_e \parallel ss_s \parallel T),$$

where

$$T = H(\text{LABEL} \parallel pk_B \parallel m_1 \parallel m_2)$$

is a public transcript hash containing the protocol labels, certified public keys, ephemeral public key, and ciphertexts.

The protocol provides unilateral authentication of the responder. Since the static DKEM contribution is obtained using B 's certified long-term DKEM public key and can be recovered only by the holder of sk_B , the initiator obtains assurance that the responder participating in the exchange is the certified owner of the long-term decapsulation key. The ephemeral DKEM contribution, generated using A 's session-specific public key, ensures that the session key also depends on a fresh initiator-side key material.

The proposed KEM-based unilateral AKE follows the Canetti-Krawczyk (CK) security model, ensuring weak Perfect Forward Secrecy (wPFS) [13, 14] under the IND-CCA security of the underlying KEM, as well as the correctness of decapsulation and the pseudorandomness of the final KDF. The transcript hash binds the session key to the identities, public keys, and exchanged ciphertexts, preventing key-misbinding and re-encapsulation-style unknown-key-share attacks.

This construction gives a bridge between an improved reconciliation-based Ding-type primitive and a deployable authenticated key exchange protocol. It maintains the algebraic foundation of Module-LWE, uses a KEM interface compatible with modern post-quantum

protocol design, and provides a simple two-pass unilateral AKE structure suitable for secure channel establishment.

1.3 Design Objectives

Security. The main objective of this proposal is to obtain a post-quantum authenticated key exchange protocol whose security can be reduced to the security of the underlying DKEM and the transcript-bound key derivation function. The protocol includes an ephemeral KEM key pair generated by the initiator for each session. The responder encapsulates this ephemeral public key, and the resulting shared secret is included in the final KDF. This design ensures that the final session key is not derived solely from the responder’s long-term KEM secret. Consequently, later exposure of sk_B does not, by itself, reveal past session keys, provided that the initiator’s ephemeral secret key was not exposed during the session. The long-term responder KEM key provides unilateral authentication, while the ephemeral initiator KEM key provides freshness. The target security model is the CK model with weak perfect forward secrecy [13, 14].

Negligible failure probability. The underlying DKEM is derived from a reconciliation-based key exchange DKE, so correctness may fail with small probability. The parameters must therefore ensure that the reconciliation failure probability δ is negligible. We use a highly refined reconciliation mechanism that utilizes fine-grained signal processing. This architectural enhancement effectively eliminates the higher failure rates observed in early Ding Key Exchange variants (e.g. 2^{-60}), reducing the reconciliation failure probability to cryptographically negligible levels (e.g. $\leq 2^{-131.7}$).

Authentication. The protocol is designed for unilateral authentication. The responder B owns a certified long-term KEM public key pk_B and secret key sk_B . The initiator derives one KEM contribution using pk_B , and only the legitimate responder can recover the corresponding shared secret by decapsulation. This method authenticates B in a way that differs from digital signature approaches.

Robust algebraic foundations and ecosystem compatibility. The underlying KEM is DKEM, which is derived from a module-lattice version of Ding Key Exchange [5, 6]. Its algebraic structure is based on modules R_q^k , seed-generated public matrices, small secret and error distributions, and reconciliation. This makes the construction compatible with the design principles of ML-KEM. Such compatibility is useful for parameter selection, implementation comparison, and standardization-oriented evaluation.

Efficiency and implementability. The two-pass protocol is designed to be efficient in both communication and computation. The initiator sends an ephemeral KEM public key and a

ciphertext associated with the responder’s long-term KEM key. The responder replies with a ciphertext associated with the initiator’s ephemeral KEM key. The authenticated key exchange ADKEX consists mainly of KEM encapsulation, decapsulation, transcript hashing, and KDF computation. Module arithmetic can be implemented using optimized polynomial operations and public matrix generation from a seed.

Transcript binding and identity binding. The final KDF includes a transcript hash T that binds the session key to the certified responder public key, the initiator’s ephemeral public key, the KEM ciphertexts, and the protocol labels. This prevents identity-misbinding and unknown-key-share patterns [15, 16] in which two parties derive the same raw KEM secret but associate it with different protocol contexts. Transcript binding also mitigates re-encapsulation attacks [17], even if the underlying KEM lacks key-binding.

Simplicity and modularity. The construction separates the key-establishment mechanism from the AKE layer. The underlying DKEM provides encapsulation and decapsulation; the AKE layer determines how long-term and ephemeral KEM contributions are combined; the transcript hash and KDF derive the final session key. This modular structure simplifies the proof and allows independent replacement or optimization of the KEM, transcript hash, or KDF.

Suitability for standardization-oriented evaluation. A proposal for an AKE that focuses on standardization should include clear algorithms, well-defined message formats, testable correctness conditions, and modular security assumptions. The proposed protocol consists of a few key components: DKEM, a transcript hash, and a Key Derivation Function (KDF). The probability of correctness failure is explicitly stated, and the protocol provides natural test vectors for key generation, encapsulation, decapsulation, transcript hashing, and final key derivation. These characteristics make the construction suitable for evaluation within the realm of commercial cryptography standardization.

2 Algorithm Description

2.1 Preliminaries

In this subsection we describe the algebraic setting, define various notations and the noise distributions used throughout the scheme, fix the public parameters and notation, and describe the generation of the public matrix $\mathbf{A}$ from a seed.

2.1.1 Ring, Modules, and Notation

Base field and modulus. Let q be an odd prime. We write $\mathbb{Z}_q$ for the finite field with q elements, and we identify it with the set of integers $\{-\frac{q-1}{2}, \dots, 0, \dots, \frac{q-1}{2}\}$ when discussing

magnitudes or norms. All additions and multiplications of coefficients are taken modulo q unless otherwise stated. In this specification, $q = 3329$ for the two firsts levels of security, and is incremented to $q = 7681$ for the last security level.

Polynomial ring. Consider the polynomial ring

$$R_q = \mathbb{Z}_q[X]/(X^n + 1),$$

where $n = 256$ for the first two security levels and $n = 512$ for the third. An element $a \in R_q$ is represented by a polynomial

$$a(X) = a_0 + a_1X + \cdots + a_{n-1}X^{n-1} \quad \text{with } a_i \in \mathbb{Z}_q,$$

where polynomials are reduced modulo $X^n + 1$, and coefficient-wise modulo q . We will often identify a with the coefficient vector $(a_0, \dots, a_{n-1}) \in \mathbb{Z}_q^n$.

Modules and matrices over R_q . For $k \geq 1$, we write R_q^k for the set of column vectors

$$\mathbf{v} = (v_1, \dots, v_k)^\top \quad \text{with } v_i \in R_q.$$

Matrices over R_q are denoted by bold uppercase letters: $\mathbf{A} \in R_q^{m \times k}$ is an $m \times k$ matrix whose entries are in R_q . Matrix-vector multiplication ($\mathbf{A}\mathbf{s} \in R_q^m$) and vector addition are defined in the natural way using ring operations in R_q .

Elements of R_q^k and $R_q^{m \times k}$ can be equivalently viewed as arrays of polynomials or as arrays of length- n coefficient vectors, depending on the context.

Inner products and norms. For $\mathbf{u}, \mathbf{v} \in R_q^k$, we define their R_q -valued inner product as:

$$\langle \mathbf{u}, \mathbf{v} \rangle = \mathbf{u}^\top \mathbf{v} = \sum_{i=1}^k u_i \cdot v_i \in R_q.$$

We also denote $\mathbf{u} \cdot \mathbf{v} = \langle \mathbf{u}, \mathbf{v} \rangle$.

When needed, we endow R_q and R_q^k with coefficient-wise norms. For $a \in R_q$ with coefficient vector $(a_0, \dots, a_{n-1})$ — lifted to integers in the canonical interval —, we define

$$\|a\|_\infty := \max_{0 \leq i < n} |a_i|, \quad \|a\|_2 := \left(\sum_{i=0}^{n-1} a_i^2 \right)^{1/2}.$$

These norms extend naturally to vectors $\mathbf{v} \in R_q^k$ by viewing $\mathbf{v}$ as a length- kn integer vector.

Number Theoretic Transform (NTT). Efficient polynomial multiplication in R_q is achieved using the Number Theoretic Transform (NTT) and its inverse, as in [8]. Recall that $R_q = \mathbb{Z}_q[X]/(X^n + 1)$, with n a power of two and q an odd prime. The arithmetic depends on the factorization of $X^n + 1$. At first it was thought that full splitting was needed, that is, $2n \mid (q - 1)$, but it was proved that having quadratic splitting is as efficient and this significantly reduces the needed modulus [18]. We are on the quadratic splitting case in all our parameter sets.

Suppose that $n \mid (q - 1)$ but $2n \nmid (q - 1)$. In this case, $\mathbb{Z}_q$ contains a primitive n -th root of unity but not a primitive $2n$ -th root. Choose $\zeta \in \mathbb{Z}_q$ such that $\zeta^n = 1$, and let $r := \log_2(n/2)$.

We define the r -bit reversal map: bit_rev_r , for $n = 2^r$, which maps an unsigned r -bit value $m = \sum_{s=0}^{r-1} a_s 2^s$ to an integer as:

$$\text{bit_rev}_r(m) = \sum_{s=0}^{r-1} a_s 2^{r-1-s}, \quad a_s \in \{0, 1\}.$$

The following factorization can be expressed in bit reversal:

$$X^n + 1 = \prod_{i=0}^{n/2-1} (x^2 - \zeta^{2\text{bit_rev}_r(i)+1}).$$

For $(n, q) = (256, 3329)$, we have:

$$x^{256} + 1 = \prod_{i=0}^{127} (x^2 - \zeta^{2\text{bit_rev}_7(i)+1}).$$

In this case $\zeta = 7$ is taken, since it is the smallest of all possible values. Similarly, for $(512, 7681)$ we have:

$$x^{512} + 1 = \prod_{i=0}^{255} (x^2 - \zeta^{2\text{bit_rev}_8(i)+1}),$$

being $\zeta = 62$ the choice in this case.

Therefore, the ring $R_q := \mathbb{Z}_q[X]/(X^n + 1)$ is isomorphic to T_q , where

$$T_q = \bigoplus_{i=0}^{n/2-1} \mathbb{Z}_q[X]/(X^2 - \zeta^{2\text{bit_rev}_r(i)+1}),$$

and the NTT representation $\hat{f} \in T_q$ of $f \in R_q$ is represented as:

$$f \mapsto \hat{f} = (f \bmod (X^2 - \zeta^{2\text{bit_rev}_r(i)+1}))_{i=0}^{n/2-1}.$$

Moreover, each transform block consists of two coefficients as:

$$f \bmod (X^2 - \zeta^{2\text{bit_rev}_r(i)+1}) = \hat{f}[2i] + \hat{f}[2i+1]X.$$

In this case, the output is split into consecutive pairs, and each pair represents a polynomial of degree < 2 in a quotient ring $\mathbb{Z}_q[X]/(X^2 - \Gamma)$. Hence, the natural multiplication subroutine is a degree-1 multiplication modulo $X^2 - \Gamma$. Algebraically,

$$(a_0 + a_1X)(b_0 + b_1X) = a_0b_0 + (a_0b_1 + a_1b_0)X + a_1b_1X^2,$$

and then X^2 is replaced by Γ . This multiplication is given by the algorithm `base_mul` (Alg. 6).

The concrete algorithms we use for the computation of these transformations and operations are provided in 2.2.2

Random oracles and XOFs. Throughout the specification, we assume the availability of cryptographic hash functions and extendable-output functions (XOFs) provided by ICCS. These are used for:

- Generating the public matrix $\mathbf{A}$ from a short seed.
- Sampling secrets and errors via rejection or bit-slicing.
- Deriving session keys from shared bit strings.

We denote generic hash/XOF functions by H, G , etc.; their precise instantiation follows SM3, HMAC-SM3 and KDF-SM3. These hash functions are used in the reference implementation just for correctness verification, with no security guarantee.

2.1.2 Noise Distributions

The security and correctness of DKE relies on carefully chosen distributions for secrets and error terms, mirroring ML-KEM. At a high level, these are *small* integer-valued distributions with support concentrated around 0 and short tails.

Centered binomial distribution. Let $\eta \in \mathbb{N}$ be a small integer parameter. The centered binomial distribution CBD_η over $\mathbb{Z}$ is defined as the distribution of the random variable

$$X = \sum_{i=1}^{\eta} (b_i - b'_i),$$

where $(b_i)_{i=1}^{\eta}$ and $(b'_i)_{i=1}^{\eta}$ are independent Bernoulli random variables with parameter $1/2$. Thus, $X \in \{-\eta, \dots, \eta\}$, $\mathbb{E}[X] = 0$, and $\text{Var}(X) = \eta$.

We denote by χ_η the discrete distribution on $\mathbb{Z}_q$ obtained by reducing samples from CBD_η modulo q .

Polynomial and vector-valued distributions. We lift χ_η to polynomials in R_q by sampling each coefficient independently:

$$a \leftarrow \chi_\eta^{R_q} \quad \text{means} \quad a(X) = \sum_{i=0}^{n-1} a_i X^i \quad \text{with} \quad a_i \leftarrow \chi_\eta \text{ i.i.d. for } 0 \leq i < n.$$

Similarly, for $k > 1$ we define $\chi_\eta^{R_q^k}$ as the product distribution on R_q^k obtained by sampling each component in R_q according to $\chi_\eta^{R_q}$.

Secret and error distributions. We consider the secret and error distribution, denoted χ_η , where η depends on the target security level (see Section 2.6). For a given parameter set, sampling

$$\mathbf{s}, \mathbf{e} \leftarrow \chi_\eta^{R_q^k}$$

means that each polynomial component and each of its coefficients are chosen independently according to the χ_η distribution. This distribution is chosen so that:

- Secrets and errors are sufficiently small to ensure correctness of the reconciliation mechanism (noise remains within decoding bounds with overwhelming probability).
- At the same time, the induced Module-LWE instances are believed to be hard for current classical and quantum attacks at the targeted security levels, as evidenced by ML-KEM's analysis [19], and is further reaffirmed by the analysis in Section 5.2.

2.1.3 Public Parameters

DKE public parameters. An instance of DKE is specified by a parameter set Π defined as follows:

$$\Pi = (R_q, q, n, k, \chi_\eta, \lambda, \lambda_K, \ell, d, G, H).$$

For the first two levels of security, this may be aligned with one of the ML-KEM parameter sets.

For a fixed choice of Π , the following quantities are public and shared by all users:

- The ring

$$R_q = \mathbb{Z}_q[X]/(X^n + 1)$$

with $q = 3329$ ($q = 7681$) and $n = 256$ ($n = 512$).

- The module rank $k \in \{2, 4\}$, determining the dimension of secrets and errors:

$$\mathbf{s}, \mathbf{e} \in R_q^k.$$

- The noise distribution χ_η for secrets and errors, as described in Section 2.1.2.
- A seed length λ (e.g., $\lambda = 256$ bits) for the public matrix seed $\rho \in \{0, 1\}^\lambda$.
- Cryptographic hash and XOF functions, modeled either as concrete primitives (e.g., SM3/KDF-SM3) or as random oracles:
 - $G : \{0, 1\}^\lambda \rightarrow \{0, 1\}^*$, an XOF used to generate $\mathbf{A}$ from ρ .
 - $H : \{0, 1\}^* \rightarrow \{0, 1\}^{\lambda_K}$, a hash or KDF used to derive the final session key from the public key in DKEM.
- Signal parameter ℓ partitioning $\mathbb{Z}_q$ into 2^ℓ subsets for more fine-grained signals. This may be chosen to match the ML-KEM compression parameter d_v .
- A compression parameter d specifying how many bits are used to represent each coefficient of polynomials. This may be chosen to match the ML-KEM compression parameter d_u .

DKEM and ADKEX parameter sets. ADKEX is obtained by instantiating the KEMTLS-PDK template [10] with DKEM as both the ephemeral and the static KEM, with no extra parameters. Hence, both protocols share the same parameter space.

In DKEM, its subjacent DKE protocol uses secret/error distribution χ_η equal to a centered binomial distribution; the output length of the shared secret and the length of seeds is always equal to n (i.e., $n = \lambda = \lambda_K$), and the hash functions are provisionally provided by ICCS (namely, pseudoHash and pseudoXOF in NGCC). Hence, we only require:

$$\pi = (q, n, k, \eta, \ell, d)$$

as a collection of public parameters. In this sense, saying *ADKEX (or DKEM) is instantiated with a parameter set $\pi = (q, n, k, \eta, \ell, d)$* means that the underlying DKE algorithms are defined with respect to

$$\Pi = (R_q, q, n, k, \chi_\eta, n, \ell, d, G, H),$$

where G and H are hash functions with at least n classical security bits (provisionally, pseudoHash and pseudoXOF provided by NGCC).

For instance, we propose three instantiations:

- **ADKEX/DKEM-128:** choosing the public parameter set $\pi_{128} = (3329, 256, 2, 3, 4, 10)$.
- **ADKEX/DKEM-256:** choosing the public parameter set $\pi_{256} = (3329, 256, 4, 2, 5, 11)$.
- **ADKEX/DKEM-512:** choosing the public parameter set $\pi_{512} = (7681, 512, 4, 3, 4, 11)$.

The numbers 128, 256 and 512 refer to the target classical security bits of each instantiation.

2.1.4 Matrix Generation and Sampling Algorithms

Following ML-KEM, the public matrix $\mathbf{A} \in R_q^{k \times k}$ is generated deterministically from a short seed ρ . This allows compact representation of the global public parameter and ensures that both parties obtain identical matrix entries without transmitting the entire matrix.

At the specification level, matrix generation (Alg. 1) is described through an auxiliary procedure `SamplePoly`, which is treated as a deterministic black-box sampler parameterized by the seed and the matrix coordinates. This keeps the specification aligned with the implemented behavior while avoiding commitment to a particular low-level XOF interface.

Algorithm 1 `GenMatrix( $\rho, transposed$ )`

Input: a seed $\rho \in \{0, 1\}^\lambda$ and a flag $transposed \in \{0, 1\}$

Output: $\mathbf{A}$ if $transposed = 0$, or $\mathbf{A}^\top$ otherwise

```

1: for  $i = 0$  to  $k - 1$  do
2:   for  $j = 0$  to  $k - 1$  do
3:     if  $transposed = 1$  then
4:        $\mathbf{A}_{i,j} \leftarrow \text{SamplePoly}(\rho, i, j)$ 
5:     else
6:        $\mathbf{A}_{i,j} \leftarrow \text{SamplePoly}(\rho, j, i)$ 
7:     end if
8:   end for
9: end for
10: if  $transposed = 0$  then
11:   return  $\mathbf{A}$ 
12: else
13:   return  $\mathbf{A}^\top$ 
14: end if

```

We model `SamplePoly( $\rho, x, y$ )` as a deterministic, domain-separated procedure which, on input the public seed ρ and two coordinate values $i, j \in \{0, \dots, k - 1\}$, outputs a polynomial in R_q . For fixed inputs, the output is uniquely determined, and for uniformly random ρ the resulting coefficients are intended to be uniformly distributed over $\mathbb{Z}_q$. The concrete realization of this procedure, including how pseudorandom bytes are expanded and mapped to coefficients, is left to the implementation as long as the resulting behavior is consistent with this specification.

We also require an auxiliary sampling procedure:

Algorithm `SampleBinomialPoly $_\eta$ ( $\mathbf{b}$ )`: On input a bit string $\mathbf{b} \in \{0, 1\}^{2\eta n}$, compute a polynomial $\mathbf{s} \in R_q$ with coefficients in $[-\eta, \eta]$. If $\mathbf{b}$ is uniformly random, then the coefficients of $\mathbf{s}$ follow the centered binomial distribution χ_η . Such an algorithm is described in Algorithm 8.

For sampling an element $s \leftarrow \chi_{\eta}^{R_q^k}$, we use $\text{SampleSecret}_{\eta}$, given in Algorithm 9.

2.2 Core Functions

The following core functions are used in the ephemeral DKE protocol (Section 2.3), which is evolved to a KEM protocol in 2.4 and used for the AKE ADKEX in 2.5.

2.2.1 Reconciliation Mechanism

The reconciliation mechanism of the original Ding Key Exchange [5, 6] is adapted here using more fine-grained signals; transmitting ℓ bits per coefficient (2^ℓ signal values) to reduce the failure rate is the general key-consensus technique of [20, 21], specialized in this proposal to the ML-KEM ring. The improved reconciliation mechanism consisting of the following two functions: Signal function (Sig) and Reconciliation function (Rec). In order to define them we partition the domain in the following way:

Partitioning the signal domain. Let ℓ be a positive integer such that $2^{\ell+1} \mid (q-1)$. Define $L = \frac{q-1}{2^\ell}$, then $\frac{L}{2}$ is an integer. The signal domain $\mathbb{Z}_q = \{-\frac{q-1}{2}, \dots, \frac{q-1}{2}\}$ is partitioned into 2^ℓ smaller subdomains:

$$\begin{aligned} I_0 &= \left\{ -\frac{L}{2}, \dots, \frac{L}{2} \right\}, \\ I_1 &= \left\{ \frac{L}{2} + 1, \dots, \frac{3L}{2} \right\}, \\ &\vdots \\ I_{2^{\ell-1}} &= \left\{ \frac{q-1-L}{2} + 1, \dots, \frac{q-1}{2} \right\} \cup \left\{ -\frac{q-1}{2}, \dots, -\frac{q-1-L}{2} - 1 \right\}, \\ &\vdots \\ I_{2^\ell-1} &= \left\{ -\frac{3L}{2}, \dots, -\frac{L}{2} - 1 \right\}. \end{aligned}$$

Signal function Sig. Let $\text{Sig} : \mathbb{Z}_q \times \{0, 1\} \rightarrow \{0, 1, \dots, 2^\ell - 1\}$. For an element $x \in \mathbb{Z}_q$, first sample b randomly from $\{0, 1\}$ with equal probability. If $x - b$ lies within the interval I_w , then define the signal of x as $\text{Sig}(x, b) = w$. The purpose of this random sampling is to eliminate statistical bias in the signal function. The signal function naturally extends to $\text{Sig} : R_q \times \{0, 1\}^n \rightarrow \{0, 1, \dots, 2^\ell - 1\}^n$ by applying it to each coefficient of a polynomial. For simplicity, when we denote $\text{Sig}(x)$, we implicitly assume a random b is already sampled. This function is specified in Algorithm 2.

Algorithm 2 $\text{Sig}(f, \mathbf{b})$ (signal function over R_q)

Input: $f \in R_q$, randomness $\mathbf{b} \in \{0, 1\}^n$;**Output:** Signal value $\mathbf{w} \in \{0, 1, 2, \dots, 2^\ell - 1\}^n$.

- 1: Initialize $\mathbf{w} = \vec{0}$.
 - 2: **for** $i = 0, \dots, n - 1$ **do**
 - 3: Compute the unique $w \in \{0, 1, 2, \dots, 2^\ell - 1\}$ such that $f[i] - \mathbf{b}[i] \pmod{q} \in I_w$.
 - 4: $\mathbf{w}[i] \leftarrow w$
 - 5: **end for**
 - 6: **return** $\mathbf{w}$.
-

Reconciliation function Rec . Let $\text{Rec} : \mathbb{Z}_q \times \{0, 1, \dots, 2^\ell - 1\} \rightarrow \{0, 1\}$. Given a signal w , an element $y \in \mathbb{Z}_q$ is mapped to 0 or 1 according to the following rule: if the signal is 0, indicating y lies in the interval I_0 , then y modulo 2 is taken directly, i.e., 0 if y is even, and 1 if odd. If the signal is $w > 0$, meaning in the interval $I_w \neq I_0$, then y is first reduced by wL , which effectively rotates the interval I_w back to I_0 , and is then taken modulo 2. Formally, this is expressed as:

$$\text{Rec}(y, w) = y - wL \pmod{q} \pmod{2}.$$

The reconciliation function naturally extends to $\text{Rec} : R_q \times \{0, 1, \dots, 2^\ell - 1\}^n \rightarrow \{0, 1\}^n$ by applying it to each coefficient of a polynomial. This function is specified in Algorithm 3.

Algorithm 3 $\text{Rec}(f, \mathbf{w})$ (Extractor over R_q)

Input: $f \in R_q$, $\mathbf{w} \in \{0, 1, \dots, 2^\ell - 1\}^n$.**Output:** $\mathbf{ss} \in \{0, 1\}^n$.

- 1: **for** $i = 0, \dots, n - 1$ **do**
 - 2: $\mathbf{ss}[i] = f[i] - \mathbf{w}[i]L \pmod{q} \pmod{2}$
 - 3: **end for**
 - 4: **return** $\mathbf{ss}$.
-

The partitioned reconciliation function has the following property: If the difference between elements $x, y \in \mathbb{Z}_q$ is even and less than or equal to $(q/2)(1 - 1/2^\ell) - 2$, then based on the signal of y , the reconciliation function maps x and y to the same value. This fact, formalized in **DKEM** specification, allows both communicating parties to reconcile and obtain the same value; that is, based on one party's signal, two numbers with the same parity and close values are mapped to the same value. When $\ell = 1$, this is consistent with the original Ding Key Exchange [5, 6]. As ℓ increases, the allowable difference grows from approximately $q/4 - 2$ to close to $q/2 - 2$, meaning the signal and reconciliation functions can tolerate larger errors. The role of partitioning the signal interval is that for the signal $w = \text{Sig}(y)$, the range of $y - wL$ is compressed from the original $\{-q/4, \dots, q/4\}$ to $\{-L/2, \dots, L/2\}$. This enables the reconciliation function $\text{Rec}(x, w) = x - wL = (x - y) + (y - wL)$ to tolerate a larger difference between x and y . Consequently, the sampling range allowed during random sampling is expanded, which greatly reduces the failure probability. But this also comes at the cost that the signal leaks

ℓ bits for each coefficient.

We prove that even after we reveal the signal w the extracted bit is still perfectly unbiased.

Lemma 2.1. Let $q > 4$ be an odd integer and ℓ be a positive integer such that $2^{\ell+1} \mid (q-1)$. If x is chosen uniformly random from $\mathbb{Z}_q$, then the reconciliation function $\text{Rec}(x, w)$ defined as above is uniformly random conditioned on the signal $\text{Sig}(x) = w \in \{0, 1, \dots, 2^\ell - 1\}$.

Proof. We leave the details to the **DKEM** specification. $\square$

2.2.2 Number-Theoretic Transform (NTT)

This section provides the concrete algorithms we use to compute the operations described in subsection 2.1.1 related to the NTT. We aim to stay consistent with the formulation of [8, 7, 19].

Algorithm 4 $\text{ntt}(f)$

Input: $f \in \mathbb{Z}_q^n$, and $\zeta \in \mathbb{Z}_q$ of order n

Output: $\hat{f} \in \mathbb{Z}_q^n$

```

1:  $\hat{f} \leftarrow f$ 
2:  $r \leftarrow \log_2(n/2)$ 
3:  $i \leftarrow 1$ 
4: for  $\text{len} = n/2, n/4, \dots, 4, 2$  do
5:   for  $\text{start} = 0, 2\text{len}, 4\text{len}, \dots, n - 2\text{len}$  do
6:      $\zeta \leftarrow \zeta^{\text{bit\_rev}_r(i)} \bmod q$ 
7:      $i \leftarrow i + 1$ 
8:     for  $j = \text{start}, \text{start} + 1, \dots, \text{start} + \text{len} - 1$  do
9:        $t \leftarrow \zeta \hat{f}[j + \text{len}] \bmod q$ 
10:       $\hat{f}[j + \text{len}] \leftarrow \hat{f}[j] - t \pmod{q}$ 
11:       $\hat{f}[j] \leftarrow \hat{f}[j] + t \pmod{q}$ 
12:    end for
13:  end for
14: end for
15: return  $\hat{f}$ 

```

- Algorithm 4 and Algorithm 5 compute the forward and inverse transforms using the classical Cooley–Tukey and Gentleman–Sande butterflies [22, 23], with the bit-reversed twiddle-factor ordering of [24]. We follow the concrete formulation also adopted in [8].
- Algorithm 6 performs the degree-one multiplication in $\mathbb{Z}_q[X]/(X^2 - \Gamma)$, and Algorithm 7 assembles the pointwise product $\hat{a} \circ \hat{b}$ over the $n/2$ blocks. Stopping at quadratic factors rather than splitting $X^n + 1$ completely, which permits the smaller moduli we use, follows [18, 25].
- We write NTT and NTT^{-1} for the transforms of Algorithms 4 and 5 applied to each coordinate of a vector of polynomials, and $\circ$ for the pointwise product of Algorithm 7, extended coordinate-wise to vectors and matrices.

Algorithm 5 $\text{inv_ntt}(\hat{f})$

Input: $\hat{f} \in \mathbb{Z}_q^n$, and $\zeta \in \mathbb{Z}_q$ of order $2n/\iota$

Output: $f \in \mathbb{Z}_q^n$

```
1:  $f \leftarrow \hat{f}$ 
2:  $r \leftarrow \log_2(n/2)$ 
3:  $i \leftarrow n/\iota - 1$ 
4: for  $\text{len} = 2, 4, \dots, n/2$  do
5:   for  $\text{start} \leftarrow 0, 2\text{len}, 4\text{len}, \dots, n - 2\text{len}$  do
6:      $\zeta \leftarrow \zeta^{\text{bit\_rev}_r(i)} \bmod q$ 
7:      $i \leftarrow i - 1$ 
8:     for  $j \leftarrow \text{start}, \text{start} + 1, \dots, \text{start} + \text{len} - 1$  do
9:        $u \leftarrow f[j]$ 
10:       $v \leftarrow f[j + \text{len}]$ 
11:       $f[j] \leftarrow u + v \pmod{q}$ 
12:       $f[j + \text{len}] \leftarrow \zeta(v - u) \pmod{q}$ 
13:    end for
14:  end for
15: end for
16:  $c \leftarrow (n/2)^{-1} \pmod{q}$ 
17: return  $c \cdot f$  ▷ component-wise multiplication modulo  $q$ 
```

Algorithm 6 $\text{base_mul}(a, b, \Gamma)$

Input: $a = (a_0, a_1)$, $b = (b_0, b_1) \in \mathbb{Z}_q^2$ and $\Gamma \in \mathbb{Z}_q$

Output: $r = (r_0, r_1) = (a_0 + a_1X)(b_0 + b_1X) \bmod (X^2 - \Gamma)$

```
1:  $r_0 \leftarrow a_0b_0 + \Gamma a_1b_1 \pmod{q}$ 
2:  $r_1 \leftarrow a_0b_1 + a_1b_0 \pmod{q}$ 
3: return  $(r_0, r_1)$ 
```

Algorithm 7 $\text{ntt_mul}(\hat{a}, \hat{b})$

Input: $\hat{a}, \hat{b} \in \mathbb{Z}_q^n$, and $\zeta \in \mathbb{Z}_q$ of order n

Output: $\hat{c} \in \mathbb{Z}_q^n$ ▷ We also denote this output as $\hat{a} \circ \hat{b}$

```
1:  $r \leftarrow \log_2(n/2)$ 
2: for  $i \leftarrow 0, 1, \dots, n/2 - 1$  do
3:    $\Gamma \leftarrow \zeta^{2\text{bit\_rev}_r(i)+1} \bmod q$ 
4:    $(\hat{c}[2i], \hat{c}[2i+1]) \leftarrow \text{base\_mul}((\hat{a}[2i], \hat{a}[2i+1]), (\hat{b}[2i], \hat{b}[2i+1]), \Gamma)$ 
5: end for
6: return  $\hat{c}$ 
```

2.2.3 Compression and Decompression Functions

We define the compression function for $x \in \mathbb{Z}_q$ as:

$$\text{Compress}_d(x) = \left\lceil \frac{2^d}{q} \cdot x \right\rceil \bmod 2^d. \quad (1)$$

For $y \in \mathbb{Z}_{2^d}$, the decompression function is:

$$\text{Decompress}_d(y) = \left\lfloor \frac{q}{2^d} \cdot y \right\rfloor. \quad (2)$$

Both functions are extended to R_q and R_q^k coefficient-wise. In these cases, they are equally denoted for notation simplicity.

Remark 2.2. Compressing and decompressing the public MLWE samples is not possible in the original Ding Key Exchange [5]. This is because successively compressing and decompressing a value in $\mathbb{Z}_q$ introduces a not necessarily even error: denote

$$r(x) = x - \text{Decompress}_d(\text{Compress}_d(x)).$$

Supposing that d is sufficiently close to $\lceil \log_2 q \rceil$, $|r(x)|$ will be close to 0. However, it might be an odd number, so compressing must be avoided in [5]. In [6], submitters introduced the so-called Round, Recover and RemoveBias functions to overcome this difficulty. Our approach is slightly different. Each time we want to communicate a coefficient $x \in \mathbb{Z}_q$ preserving parity, instead of directly compressing x , we compress $\frac{1}{2}x$ (where $\frac{1}{2}$ denotes the inverse of 2 mod q) to obtain x' , and the receiver recovers $2 \cdot x' \bmod q$. It is easy to check that both properties hold: the error introduced is both small and even as long as d is sufficiently close to $\lceil \log_2 q \rceil$.

2.3 Specification of DKE

In this subsection we give a specification of DKE (Algorithms 8, 9, 10, 11 and 12), a two-pass key exchange protocol between an initiator A and a responder B . At the end of the protocol, both parties compute a shared secret $ss \in \{0, 1\}^n$. We discuss the correctness and reconciliation failure probability for this protocol in 3.3.

2.4 Specification of CCA-Secure DKEM

DKEM is constructed directly from DKE with the final key derived directly from the shared secret. This is specified in Algorithms 13, 14, 15 and 16.

Algorithm 8 SampleBinomialPoly $_{\eta}(\mathbf{b})$

Input: $\mathbf{b} \in \{0, 1\}^{2 \cdot \eta \cdot n}$.

Output: $s \in R_q$ with coefficients sampled from $\mathcal{B}_{\eta}(\mathbb{Z}_q)$, as long as $\mathbf{b}$ is uniformly random.

```
1: for  $i = 0, \dots, n - 1$  do
2:    $a \leftarrow \sum_{j=0}^{\eta-1} \mathbf{b}[j + 2 \cdot i \cdot \eta]$ 
3:    $b \leftarrow \sum_{j=0}^{\eta} \mathbf{b}[j + \eta + 2 \cdot i \cdot \eta]$ 
4:    $s[i] \leftarrow a - b$ 
5: end for
   return  $s$ .
```

Algorithm 9 SampleSecret $_{\eta}(r, nonce)$

Input: $r \in \{0, 1\}^n$, $nonce \in \mathbb{N}$

Output: $\mathbf{s} \in R_q^k$ with coefficients sampled from $\mathcal{B}_{\eta}(\mathbb{Z}_q)$.

```
1: for  $j = 0, \dots, k - 1$  do
2:    $\mathbf{b} \leftarrow \text{XOF}((r || nonce); 2 \cdot \eta \cdot n)$ 
3:    $\mathbf{s}[j] \leftarrow \text{SampleBinomialPoly}_{\eta}(\mathbf{b})$ 
4:    $++ nonce$ 
5: end for
   return  $\mathbf{s}, nonce$ 
```

Algorithm 10 DKE.Initiate($coins$)

Input: randomness $coins \in \{0, 1\}^n$.

Output: public key pk .

Output: private key sk .

```
1:  $(r, \rho) \leftarrow \text{XOF}(coins; 2n)$  ▷ expand to two pseudorandom  $n$ -bit seeds
2:  $\bar{\mathbf{A}} \leftarrow \text{GenMatrix}(\rho)$  ▷ generate matrix  $\mathbf{A} \in (\mathbb{Z}_q^n)^{k \times k}$ 
3:  $(\mathbf{s}_A, nonce) \leftarrow \text{SampleSecret}_{\eta}(r, 0)$  ▷ generate secret  $\mathbf{s}_A \in (\mathbb{Z}_q^n)^k$ 
4:  $\mathbf{e}_A \leftarrow \text{SampleSecret}_{\eta}(r, nonce)$  ▷ generate error  $\mathbf{e}_A \in (\mathbb{Z}_q^n)^k$ 
5:  $\bar{\mathbf{s}}_A \leftarrow \text{NTT}(\mathbf{s}_A)$  ▷ run NTT  $k$ -times (once for each coordinate of  $\mathbf{s}_A$ )
6:  $\bar{\mathbf{e}}_A \leftarrow \text{NTT}(\mathbf{e}_A)$  ▷ run NTT  $k$ -times (once for each coordinate of  $\mathbf{e}_A$ )
7:  $\bar{\mathbf{u}}_A \leftarrow \bar{\mathbf{A}} \circ \bar{\mathbf{s}}_A + \bar{\mathbf{e}}_A$  ▷ noisy linear system in NTT domain
8:  $pk \leftarrow \text{ByteEncode}(\bar{\mathbf{u}}_A) || \rho$ 
9:  $sk \leftarrow \text{ByteEncode}(\bar{\mathbf{s}}_A)$ 
10: return  $pk, sk$ 
```

Algorithm 11 DKE.Response($pk, coins$)

Input: public key pk . ▷ initiator's public key pk
Input: random $coins \in \{0, 1\}^{2n}$.
Output: response message c . ▷ responder's compressed data with signal
Output: shared secret $ss \in \{0, 1\}^n$.

- 1: $(\bar{\mathbf{u}}_A, \rho) \leftarrow pk$
- 2: $(\mathbf{r}, \mathbf{b}) \leftarrow coins$
- 3: $\bar{\mathbf{A}}^\top \leftarrow \text{GenMatrix}(\rho, 1)$
- 4: $\mathbf{s}_B, nonce \leftarrow \text{SampleSecret}_\eta(\mathbf{r}, 0)$
- 5: $\mathbf{e}_B \leftarrow \text{SampleSecret}_\eta(\mathbf{r}, nonce)$
- 6: $\bar{\mathbf{s}}_B \leftarrow NTT(\mathbf{s}_B)$.
- 7: $\bar{\mathbf{u}}_B \leftarrow \bar{\mathbf{A}}^\top \circ \bar{\mathbf{s}}_B$
- 8: $\mathbf{u}_B \leftarrow NTT^{-1}(\bar{\mathbf{u}}_B)$
- 9: $\mathbf{u}_B \leftarrow \mathbf{u}_B + \mathbf{e}_B$
- 10: $\mathbf{u}'_B \leftarrow \text{Compress}(\mathbf{u}_B, d_B)$. ▷ compress responder's data
- 11: $\bar{k}_B \leftarrow \bar{\mathbf{u}}_A \circ \bar{\mathbf{s}}_B$
- 12: $k_B \leftarrow NTT^{-1}(\bar{k}_B)$
- 13: $\mathbf{r} \leftarrow \text{XOF}(\mathbf{r} || nonce; 2 \cdot \eta \cdot n)$
- 14: $\mathbf{e}_B \leftarrow \text{SampleBinomialPoly}(\mathbf{r})$
- 15: $k_B \leftarrow k_B + \mathbf{e}_B$
- 16: $k_B \leftarrow 2 \cdot k_B$
- 17: $\mathbf{w} \leftarrow \text{Sig}(k_B, \mathbf{b})$ ▷ generate the signal
- 18: $c \leftarrow (\mathbf{u}'_B || \mathbf{w})$ ▷ generate response message from responder's data and signal
- 19: $ss \leftarrow \text{Rec}(k_B, \mathbf{w})$ ▷ derive the shared secret by applying error reconciliation
- 20: **return** (c, ss)

Algorithm 12 DKE.DeriveSecret(c, sk)

Input: message c . ▷ responder's compressed data with signal
Input: private key $sk = \bar{\mathbf{s}}_A$.
Output: shared secret $ss \in \{0, 1\}^n$.

- 1: $(\mathbf{u}'_B, \mathbf{w}) \leftarrow c$
- 2: $\mathbf{u}''_B \leftarrow \text{Decompress}(\mathbf{u}'_B, d_B)$
- 3: $\bar{\mathbf{u}}''_B \leftarrow NTT(\mathbf{u}''_B)$
- 4: $\bar{k}_A \leftarrow \bar{\mathbf{u}}''_B \circ \bar{\mathbf{s}}_A$
- 5: $k_A \leftarrow 2 \cdot NTT^{-1}(\bar{k}_A)$
- 6: $ss \leftarrow \text{Rec}(k_A, \mathbf{w})$ ▷ derive the shared secret by applying error reconciliation
- 7: **return** ss

Algorithms. The following algorithms give the full CCA-secure DKEM interface.

Algorithm 13 DKEM.KeyGen()

Output: encapsulation key ek .

Output: decapsulation key dk .

- 1: $coins \leftarrow \{0, 1\}^n$
 - 2: $pk, sk \leftarrow \text{DKE.Initiate}(coins)$
 - 3: $rej \leftarrow \{0, 1\}^n$ $\triangleright$ randomness for implicit rejection in case of failure
 - 4: $dk = (sk || pk || rej)$
 - 5: **return** $(ek = pk, dk)$
-

Algorithm 14 DKEM.Internal($ek, coins$)

Input: encapsulation key $ek = (... || \rho)$.

Input: randomness $coins \in \{0, 1\}^n$.

Output: ciphertext ct .

Output: shared key $K \in \{0, 1\}^n$.

- 1: $\rho \leftarrow ek$ $\triangleright \rho$ comes from the last n bits of ek
 - 2: $r \leftarrow \text{XOF}((coins || \rho); 2n)$
 - 3: $c_1, ss \leftarrow \text{DKE.Response}(ek, r)$
 - 4: $c_2 \leftarrow coins \oplus ss$ $\triangleright$ mask the randomness by the shared secret
 - 5: $ct = (c_1 || c_2)$
 - 6: $K \leftarrow \text{KDF}(ss; n)$ $\triangleright$ generate the shared key from shared secret
 - 7: **return** (ct, K)
-

Algorithm 15 DKEM.Encaps(pk)

Input: encapsulation key ek .

Output: ciphertext ct .

Output: shared key $K \in \{0, 1\}^n$.

- 1: $coins \leftarrow \{0, 1\}^n$
 - 2: $ct, K \leftarrow \text{DKEM.Internal}(ek, coins)$. $\triangleright$ encrypt the random coins and derive the shared key
 - 3: **return** (ct, K)
-

2.5 Specification of ADKEX

The ADKEX authenticated key exchanged is specified in Algorithms 17, 18, 19, 20 and 21, based on [10]. The scheme is presented in a visual way in Figure 1. The sizes for messages and randomness used in ADKEX are indicated in Table 1.

Notation. The ADKEX parameter N is inherited from the underlying DKEM. The byte-lengths PKBITS , SKBITS and CTBITS are those of the DKEM (CCA) scheme as described in the **DKEM** submission. ADKEX uses DKEM for both the ephemeral and the static KEM; the randomness

Algorithm 16 DKEM.Decaps(dk, ct)

Input: decapsulation key dk .

Input: ciphertext $ct = (c_1 || c_2)$.

Output: shared key $K \in \{0, 1\}^n$.

```
1:  $(sk, ek, rej) \leftarrow dk$ 
2:  $(c_1, c_2) \leftarrow ct$ 
3:  $ss \leftarrow \text{DKE.DeriveSecret}(c_1, sk)$  ▷ derive the shared secret of key exchange
4:  $coins \leftarrow c_2 \oplus ss$  ▷ unmask to recover the randomness
5:  $(ct', K) \leftarrow \text{DKEM.Internal}(ek, coins)$  ▷ re-construct using the derived randomness
6:  $\bar{K} \leftarrow \text{KDF}(rej || c_2; n)$ 
7: if  $ct \neq ct'$  then
8:   return  $\bar{K}$  ▷ if ciphertexts do not match, “implicitly reject”
9: end if
10: return  $K$ 
```

consumed by each underlying call is:

$$|\rho_{\text{DKEM.KeyGen}}| = 2N \qquad |\rho_{\text{DKEM.Encaps}}| = N.$$

The hash and key-derivation functions H , KDF are instantiated with SM3-256 for $N = 256$ and with the SM3-based pseudoXOF truncated to $N = 512$ bits for ADKEX-512. The per-parameter-set ASCII domain-separation label is $\text{LABEL} = \text{"ADKEX-KEMTLS-}\ell\text{"}$ with $\ell \in \{128, 256, 512\}$. The transcript binder is:

$$T = H(\text{LABEL} || pk_B || m_1 || m_2; N).$$

The empty string is denoted ε and bit-string concatenation by $||$.

Algorithm 17 ADKEX.init_b(ρ)

▷ Responder setup

Input: randomness $\rho \in \{0, 1\}^{2N}$.

Output: responder long-term encapsulation key $pk_B \in \{0, 1\}^{\text{PKBITS}}$.

Output: decapsulation key $sk_B \in \{0, 1\}^{\text{SKBITS}}$.

Output: state $st_B = \varepsilon$.

```
1:  $(pk_B, sk_B) \leftarrow \text{DKEM.KeyGen}(\rho)$  ▷ static CCA key pair
2:  $st_B \leftarrow \varepsilon$ 
3: return  $(pk_B, sk_B, st_B)$ 
```

Algorithm 18 ADKEX.generate_pass1_msg_a(pk_B, ρ_1, ρ_3)

Input: responder long-term encapsulation key $pk_B \in \{0, 1\}^{\text{PKBITS}}$ $\triangleright$ pre-distributed.
Input: randomness $\rho_1 \in \{0, 1\}^{2 \cdot N}$ for the ephemeral key pair.
Input: randomness $\rho_3 \in \{0, 1\}^N$ for the static encapsulation.
Output: pass-1 message $m_1 \in \{0, 1\}^{\text{PKBITS} + \text{CTBITS}}$.
Output: updated state $st_A \in \{0, 1\}^{N + \text{SKBITS} + \text{PKBITS} + \text{CTBITS}}$.
1: $(M_1, sk_e) \leftarrow \text{DKEM.KeyGen}(\rho_1)$ $\triangleright$ ephemeral CCA key pair
2: $(ct_S, ss_s) \leftarrow \text{DKEM.Encaps}(pk_B \| \rho_3)$ $\triangleright$ static CCA encapsulation, $|ss_s| = N$
3: $m_1 \leftarrow M_1 \| ct_S$
4: $st_A \leftarrow sk_e \| ss_s \| m_1$ $\triangleright m_1$ kept in state to compute T in derive_ss_a
5: **return** (m_1, st_A)

Algorithm 19 ADKEX.generate_pass2_msg_b(m_1, pk_B, sk_B, ρ_2) $\triangleright$ Final pass

Input: pass-1 message $m_1 \in \{0, 1\}^{\text{PKBITS} + \text{CTBITS}}$.
Input: responder long-term encapsulation key $pk_B \in \{0, 1\}^{\text{PKBITS}}$.
Input: decapsulation key $sk_B \in \{0, 1\}^{\text{SKBITS}}$.
Input: randomness $\rho_2 \in \{0, 1\}^N$ for the ephemeral encapsulation.
Output: pass-2 message $m_2 \in \{0, 1\}^{\text{CTBITS}}$.
Output: updated state $st_B \in \{0, 1\}^{3 \cdot N}$.
1: $(M_1, ct_S) \leftarrow m_1$
2: $(M_2, ss_e) \leftarrow \text{DKEM.Encaps}(M_1 \| \rho_2)$ $\triangleright$ ephemeral CCA encapsulation, $|ss_e| = N$
3: $ss_s \leftarrow \text{DKEM.Decaps}(sk_B, ct_S)$ $\triangleright |ss_s| = N$
4: $m_2 \leftarrow M_2$
5: $T \leftarrow H(\text{LABEL} \| pk_B \| m_1 \| m_2; N)$
6: $st_B \leftarrow ss_e \| ss_s \| T$
7: **return** (m_2, st_B) $\triangleright$ kex finished on B 's side

Algorithm 20 ADKEX.derive_ss_a(m_b, st_A, pk_B)

Input: last responder message $m_2 \in \{0, 1\}^{\text{CTBITS}}$ $\triangleright$ i.e. $m_b = m_2 = M_2$.
Input: state $st_A \in \{0, 1\}^{\text{SKBITS} + N + \text{PKBITS} + \text{CTBITS}}$.
Input: responder long-term encapsulation key $pk_B \in \{0, 1\}^{\text{PKBITS}}$ $\triangleright$ pre-distributed.
Output: shared key $ss \in \{0, 1\}^N$.
1: $(sk_e, ss_s, m_1) \leftarrow st_A$
2: $ss_e \leftarrow \text{DKEM.Decaps}(sk_e, m_2)$ $\triangleright |ss_e| = N$, includes implicit rejection
3: $T \leftarrow H(\text{LABEL} \| pk_B \| m_1 \| m_2; N)$
4: $ss \leftarrow \text{KDF}(ss_e \| ss_s \| T; N)$
5: **return** ss

Algorithm 21 ADKEX.derive_ss_b(m_a, st_B, sk_B)

Input: last initiator message $m_a \in \{0, 1\}^{\text{PKBITS}+\text{CTBITS}}$ $\triangleright$ i.e. $m_a = m_1$.

Input: state $st_B \in \{0, 1\}^{3 \cdot N}$ as set by ADKEX.generate_pass2_msg_b.

Input: responder long-term decapsulation key $sk_B \in \{0, 1\}^{\text{SKBITS}}$ $\triangleright$ ignored here, consumed in pass 2.

Output: shared key $ss \in \{0, 1\}^N$.

- 1: $(ss_e, ss_s, T) \leftarrow st_B$
 - 2: $ss \leftarrow \text{KDF}(ss_e \parallel ss_s \parallel T; N)$
 - 3: **return** ss
-

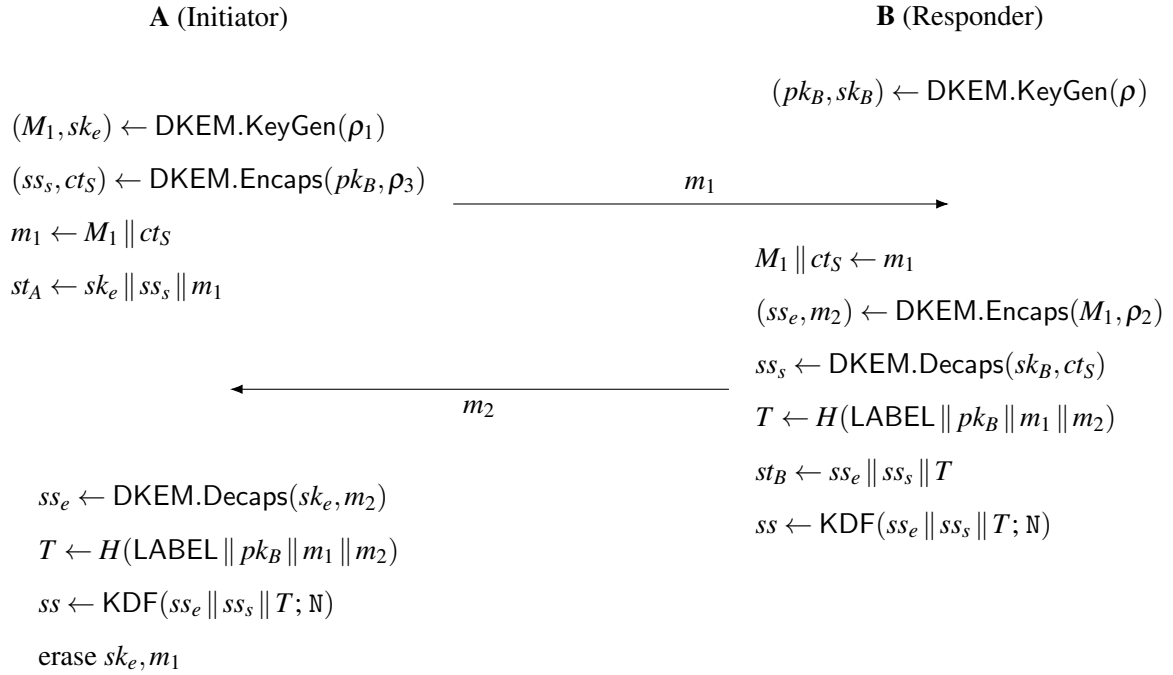


Figure 1: Two-pass authenticated key exchange ADKEX following the KEMTLS-PDK [10] pattern. Both the ephemeral and the static KEM are DKEM (CCA). The initiator A holds no long-term key; the responder's static encapsulation key pk_B is pre-distributed. The transcript $T = H(\text{LABEL} \parallel pk_B \parallel m_1 \parallel m_2; N)$ is passed into the KDF on both sides to defend against re-encapsulation attacks [17] even when DKEM provides no key-binding property.

	ADKEX-128	ADKEX-256	ADKEX-512
N	256	256	512
PKBITS/8	800	1568	3392
SKBITS/8	1600	3136	6784
CTBITS/8	800	1600	3136
$ m_1 /8$	1600	3168	6528
$ m_2 /8$	800	1600	3136
total transcript (BYTES)	2400	4768	9664
$ st_A _{\max}/8$	3232	6336	13376
$ st_B _{\max}/8$	96	96	192
$ \rho /8$ for init_b	64	64	128
$ \rho_1 /8$ for pass 1 (DKEM.KeyGen)	64	64	128
$ \rho_3 /8$ for pass 1 (DKEM.Encaps)	32	32	64
$ \rho_2 /8$ for pass 2 (DKEM.Encaps)	32	32	64
H, KDF instance	SM3-256	SM3-256	pseudoXOF-512

Table 1: Concrete sizes for the different messages and used randomness in the three variants of ADKEX. All values are in bytes unless otherwise indicated.

2.6 Parameter Sets

We summarize the parameter sets of ADKEX in Table 2.

Name	n	q	k	η	ℓ	d	Failure prob.	NGCC level
ADKEX-128	256	3329	2	3	4	10	$2^{-131.7}$	1
ADKEX-256	256	3329	4	2	5	11	$2^{-180.2}$	2
ADKEX-512	512	7681	4	3	4	11	$2^{-166.0}$	3

Table 2: Summary of parameter sets for ADKEX. The failure probability column has been calculated as described in Section 3.3, and using the scripts in [26].

3 Design Rationale

3.1 The mathematical hard problem

The security of ADKEX rests on the same hardness assumption as its single building block, the CCA-secure KEM DKEM: the Module Learning With Errors (MLWE) problem. ADKEX introduces no new algebraic structure and no new computational assumption beyond MLWE; long-term authentication is obtained generically, by simplyfing the KEMTLS-PDK template [10] with DKEM as both the ephemeral and the static KEM. We refer to the **DKEM** specification for the formal statement of MLWE and its relation to the security of the underlying DKE and DKEM.

3.2 Criteria for Parameter Selection

parameters of its own beyond the choice of which DKEM variant instantiates each security level and the symmetric primitive used for H and KDF. The key-exchange parameters are exactly those analyzed in the **DKEM** specification. Binding both KEMs to the same variant keeps the two MLWE instances at a common security level and avoids a mismatch between the ephemeral and static components. The shared secret has length N bits, matching the targeted classical security level (Section 2.1 of [27]); accordingly H and KDF are instantiated with SM3-256 at the $N = 256$ levels and with the SM3-based pseudoXOF truncated to 512 bits for ADKEX-512, where a 512-bit output is required (Table 1).

3.3 Failure probability analysis.

A run of ADKEX invokes DKEM twice along the path that affects correctness: the responder encapsulates against the initiator’s ephemeral key and the initiator decapsulates it, and the

initiator encapsulates against the responder’s static key and the responder decapsulates it. Each of these two encapsulation/decapsulation pairs agrees on its shared secret except with the reconciliation failure probability δ of the underlying **DKEM** variant, established in the failure-probability analysis of the DKEM specification. Since both parties derive the session key from ss_e , ss_s and the transcript binder T , a single failure in either pair makes the two sides derive different keys. By a union bound the probability that an honest ADKEX session fails to agree on a key is therefore at most 2δ , i.e. at most $2^{-131.7}$, $2^{-180.2}$ and $2^{-166.0}$ for ADKEX-128, ADKEX-256 and ADKEX-512 respectively, which remains cryptographically negligible. Failure-driven attacks are treated in Section 5.2.5.

3.4 Other considerations

Why DKEM (CCA) and not the bare DKE core (CPA). The ephemeral key is decapsulated once per session, so IND-CCA1 would suffice as in [10]. We assume full IND-CCA on it anyway by reusing the single DKEM primitive — conservative, could be tightened to IND-CCA1 —. Static KEM needs IND-CCA in both (key reuse). ADKEX therefore instantiates both the static and the ephemeral KEM with the full DKEM rather than the bare DKE core. By contrast, the **DKEX** protocol (submitted as a separate submission to NGCC) can use the CPA core DKE directly, because its key exchange is purely ephemeral and long-term authentication is supplied by a signature.

Transcript binding and domain separation. **DKEM** deliberately leaves the binding of the public key and ciphertext to the shared secret to the protocols built on top of it. ADKEX takes up this responsibility at the instantiation level: rather than relying on any key-binding property of DKEM, it derives the session key as $\text{KDF}(ss_e \parallel ss_s \parallel T)$, where $T = H(\text{LABEL} \parallel pk_B \parallel m_1 \parallel m_2)$ binds both KEM messages and the responder’s static key under a fixed domain-separation label. This is what lets ADKEX resist re-encapsulation attacks [17] even though DKEM exposes no key-binding guarantee; the security consequences are analysed in Section 5.2.5.

4 Performance Evaluation

Implementation scope and provenance. ADKEX is implemented as a two-pass authenticated key-exchange protocol using DKEM for the responder’s long-term and ephemeral KEM operations. The Reference Implementation links the portable C DKEM arithmetic, while the Performance Optimization Version links the AVX2 DKEM backend. Both versions expose the same ADKEX interface and protocol-message formats.

Reference implementation. The portable implementation executes the complete ADKEX exchange using the reference C polynomial arithmetic, NTT, modular reduction, sampling,

packing, and reconciliation routines inherited from the DKEM implementation (Tables 3 and 4).

Parameter set	Avg. time (μ s)	Avg. cycles	Throughput (ops/s)	Static image (B)	Peak RSS (KB)
ADKEX-128	517.10	1 245 094	1 933.86	40 230	1 876
ADKEX-256	1 286.29	3 099 054	777.43	41 446	1 876
ADKEX-512	4 406.24	10 691 576	226.95	43 054	1 876

Table 3: Reference Implementation full-exchange performance. Performance values are arithmetic means.

Parameter set	Long-term pk	Long-term sk	State A	State B	Message 1	Message 2	Total transmitted	ss
ADKEX-128	800	1 600	3 232	96	1 600	800	2 400	32
ADKEX-256	1 568	3 136	6 336	96	3 168	1 600	4 768	32
ADKEX-512	3 392	6 784	13 376	192	6 528	3 136	9 664	64

Table 4: Reference Implementation communication and storage sizes. All sizes are reported in bytes.

Benchmarking configuration. Measurements were collected on an Intel Core i9-10885H at 2.40 GHz with 64 GiB of RAM, running Ubuntu 24.04 under Microsoft WSL2 (kernel 6.x). The implementation was compiled with GCC 13.3.0 using `-std=c99`, `-Wpedantic`, `-Wall`, `-Wextra`, `-O2`, `-fcommon`, `-DDKE_FORCE_SCALAR`, `-DDKE_MODE=<128|256|512>`, `-DDKE_HASH=0`, and `-DDKE_RANDOM=0`. Each entry is the arithmetic mean of 128 complete exchanges measured on a single core. Execution time was measured with `clock_gettime(CLOCK_MONOTONIC)`, and CPU cycles were measured with RDTSC; throughput is derived from the average execution time. Static image size is `text+data+bss` as reported by GNU `size`, and peak RSS is reported by `getrusage(RUSAGE_SELF)`.

AVX2 Performance Optimization implementation. The optimized AVX2 implementation preserves the high-level two-pass ADKEX logic and message formats while replacing the DKEM arithmetic backend with AVX2-optimized polynomial arithmetic. The linked backend includes the GNU assembly routines `fq.S`, `ntt.S`, `invntt.S`, `basemul.S`, and `shuffle.S` (Table 5). Communication and storage sizes are unchanged from the Reference Implementation.

Parameter set	Avg. time (μ s)	Avg. cycles	Throughput (ops/s)	Static image (B)	Peak RSS (KB)
ADKEX-128	297.87	777 327	3 357.17	128 509	1 876
ADKEX-256	903.70	2 188 102	1 106.56	130 390	1 876
ADKEX-512	3 364.96	8 809 006	297.18	126 303	1 876

Table 5: AVX2 Performance Optimization Version full-exchange results. Performance values are arithmetic means.

Benchmarking configuration. The platform, timing method, sample count, and memory definitions are identical to those used for Table 3. The implementation was compiled with GCC 13.3.0 using `-O3`, `-march=x86-64`, `-mavx2`, `-mtune=native`, `-flto`, `-fomit-frame-pointer`, `-std=c99`, `-Wpedantic`, `-Wall`, `-Wextra`, `-fcommon`, `-DDKE_HASH=0`, and `-DDKE_RANDOM=0`, together with the linked GNU AVX2 assembly sources.

ARMv8-A Performance Optimization implementation. The optimized NEON implementation preserves the high-level two-pass ADKEX logic and message formats while replacing the DKEM arithmetic backend with NEON-optimized polynomial arithmetic. These links provide hand-optimized routines for NTT/INTT, base multiplication, and polynomial operations. The reference SM3/pseudoXOF is used to generate pseudorandom number in ADKEX. Note that this arm64 platform has no stable user-space cycle counter (PMU restricted), so average cycles are not reported; average wall-clock microseconds (μ s) are authoritative and used in Table 6.

Parameter set	Avg. time (μ s)	Throughput (ops/s)	Peak RSS (KB)
ADKEX-128	281.64	3 550.63	1 776
ADKEX-256	806.69	1 239.63	1 776
ADKEX-512	3 025.32	330.54	1 776

Table 6: ARMv8-A Performance Optimization Version full-exchange results. Throughput is derived from the average time.

Benchmarking configuration. Measurements were collected on an ARMv8.2-A architecture, single-core measurement. The implementation was built with the GNU GCC 13 using `-O3`, `-march=armv8.2-a`, `-flto`, `-fomit-frame-pointer`, `-std=c99`, `-Wpedantic`, `-Wall`, `-Wextra`, `-fcommon`, `-DDKE_USE_AARCH64`, `-DDKE_USE_AARCH64_NATIVE`, `-DDKE_HASH=0`, and `-DDKE_RANDOM=0`. The average time entries are arithmetic means of 128 measurements. Each iteration is timed with `clock_gettime(CLOCK_MONOTONIC)`. Peak RSS is process peak resident memory reported by `getrusage(RUSAGE_SELF)`.

ARM Cortex-M4 Performance Optimization implementation. The optimized ARM Cortex-M4 implementation preserves the high-level two-pass ADKEX logic and message formats while replacing the DKEM arithmetic backend with Cortex-M4-optimized polynomial arithmetic. The linked backend includes the Cortex-M4 assembly routines for NTT/INTT, base multiplication, modular reduction, polynomial operations, matrix accumulation, and SM3 (Table 7). Communication and storage sizes are unchanged from the Reference Implementation.

Parameter set	init_b cycles	pass1 cycles	pass2 cycles	derive cycles
ADKEX-128	53 793	116 727	156 123	95 441
ADKEX-256	167 126	349 893	424 704	231 471
ADKEX-512	588 118	1 239 101	1 453 401	810 987

Table 7: Cortex-M4 stage cycle counts from the QEMU self-test harness. These results are additional embedded-platform evidence.

Benchmarking configuration. The Cortex-M4 implementation was built with `arm-none-eabi-gcc 14.2.1` using `-mcpu=cortex-m4`, `-mthumb`, `-mfloat-abi=hard`, `-mfpv4-sp-d16`, `-O2`, `-ffunction-sections`, `-fdata-sections`, `-std=gnu99`, `-nostdlib`, and `-Wl,--gc-sections`. It was executed under QEMU on the `netduinoplus2` Cortex-M4 target. Cycle counts were obtained from the 24-bit SysTick down-counter and are the minimum values observed over five complete self-test iterations. The implementation uses deterministic inputs, Plantard arithmetic, matrix accumulation, and the SM3 backend. The Cortex-M4 results are outside the ARMv8.2-A guideline scope and are provided as additional evidence only.

5 Security Statements and Analyses

5.1 Theoretical security

In this section, we first define the underlying security model, namely the Canetti-Krawczyk (CK) security model with weak Perfect Forward Secrecy (wPFS) [13, 14], and prove that the proposed authenticated key exchange ADKEX is secure under the CK security model with wPFS.

5.1.1 Security Model

We provide a CK security model for a two-pass unilateral DKEM-based AKE.

Parties, static keys, and sessions. Let $\mathcal{U}$ be a set of parties. Each party is a probabilistic polynomial-time interactive machine. A party may own a long-term static key pair. In ADKEX, only the responder B owns a static DKEM key pair (pk_B, sk_B) for unilateral authentication.

A *session* is one local execution of the protocol at one party. A session has:

- an owner;
- an intended peer;
- a role, initiator I or responder R ;
- a public message transcript;
- a session state st , containing session-specific secret information.

A session is *completed* if its owner outputs a session key.

For ADKEX, an initiator session has identifier

$$\text{sid} = (\Pi, I, A, B, m_1, m_2),$$

and a responder session has an identifier

$$\text{sid} = (\Pi, R, B, A, m_1, m_2),$$

where Π represents the protocol identifier, and (m_1, m_2) are the protocol messages exchanged by parties A and B respectively.

Matching sessions. An initiator session

$$\text{sid} = (\Pi, I, A, B, m_1, m_2)$$

and a responder session

$$\text{sid} = (\Pi, R, B, A, m_1, m_2)$$

are matching if they have opposite roles, opposite owner/peer identities, and the same public protocol messages (m_1, m_2) . We write $\overline{\text{sid}}$ for the matching session of sid , when it exists.

Adversary queries. The adversary possesses the capability to dominate the network, initiating sessions and transmitting custom messages through send queries. Moreover, it can access specific secrets using reveal queries.

$\text{send}(\cdot)$: initiates a session or gets protocol messages from the parties.

$\text{Reveal}(\text{sid})$: if session sid is completed, returns its accepted session key.

$\text{StateReveal}(\text{sid})$: if the session sid is not completed, it returns its session state, including ephemeral secret keys and intermediate computations, but not long-term static secrets.

$\text{Corrupt}(P)$: returns the long-term static secret key of party P , if such a key exists. For ADKEX, the relevant corruption is $\text{Corrupt}(B)$, which returns sk_B .

Freshness. Let sid^* be a completed session owned by an honest party with an honest peer. If a matching session exists, denote it by $\overline{\text{sid}^*}$. The session sid^* is *fresh* if none of the following conditions holds:

- The adversary asks $\text{Reveal}(\text{sid}^*)$.
- A matching session $\overline{\text{sid}^*}$ exists and the adversary makes one of the following queries
 - $\text{Reveal}(\overline{\text{sid}^*})$,
 - $\text{StateReveal}(\text{sid}^*)$ or
 - $\text{StateReveal}(\overline{\text{sid}^*})$.
- No matching session exists and the adversary asks $\text{StateReveal}(\text{sid}^*)$.

Security experiment. The AKE experiment in the CK model proceeds as follows.

- The challenger initializes parties and static keys.
- The adversary adaptively makes send, Reveal, StateReveal, and Corrupt queries.
- At some point, the adversary chooses a completed fresh session sid^* and asks $\text{Test}(\text{sid}^*)$.
- The challenger samples $b \leftarrow \{0, 1\}$. If $b = 0$, it returns the real accepted session key of sid^* . If $b = 1$, it returns a uniformly random string of the same length.
- The adversary continues making allowed queries, preserving freshness, and eventually outputs a guess b' .

The adversary wins if $b' = b$ and sid^* remains fresh.

CK security with weak perfect forward secrecy. An AKE protocol is secure in the CK model with weak perfect forward secrecy if the following conditions hold.

- **Correctness.** If two honest parties complete matching sessions, then, except with negligible probability, they output the same session key.
- **Key indistinguishability.** For any PPT adversary $\mathcal{A}$,

$$\text{Adv}_{\Pi}^{\text{CK}}(\mathcal{A}) = \left| \Pr[b' = b] - \frac{1}{2} \right|$$

is negligible for every fresh test session sid^* in the following cases:

- **No matching session.** No matching session $\overline{\text{sid}^*}$ exists.
- **wPFS.** A matching session $\overline{\text{sid}^*}$ exists, and the adversary is additionally given the long-term static secret keys of the owners of sid^* and $\overline{\text{sid}^*}$, while the test-session and matching-session states remain unrevealed by freshness.

For the unilateral protocol ADKEX, only B has a long-term static authentication secret. Therefore, the wPFS case for ADKEX means that the matching session exists and the adversary may learn B 's static secret key sk_B . In contrast, it may not learn the initiator's ephemeral secret key sk_e for the tested session.

5.1.2 The wPFS security for ADKEX.

Let q_s be an upper bound on the number of completed matching sessions that may become the test session. Let q_H be the number of random-oracle queries to the KDF. Let n be the shared secret key length, so $|\mathcal{K}| = 2^n$.

Theorem 5.1 (wPFS security of ADKEX). Assume:

- the underlying DKEM is IND-CCA secure;
- the DKEM has a correctness error at most δ ;
- the KDF is modeled as a random oracle.

Then, for every adversary $\mathcal{A}$ for the weak-forward-secrecy of ADKEX, activating at most q_s candidate matching sessions and making at most q_H queries to KDF, there exists an IND-CCA adversary $\mathcal{B}$ against the DKEM such that

$$\text{Adv}_{\text{ADKEX}}^{\text{wPFS}}(\mathcal{A}) \leq 2q_s \cdot \text{Adv}_{\text{DKEM}}^{\text{ind-cca}}(\mathcal{B}) + \frac{q_H}{2^n} + 2\delta.$$

Proof. We prove the case of wPFS, where the static secret key can be revealed to the adversary. The non-matching case can be proven similarly.

Correctness. If two honest parties complete matching sessions, then, except with probability at most 2δ , they output the same session key.

Game 0: real wPFS experiment. Let G_0 be the real wPFS experiment for ADKEX. Let Succ_0 be the event that $\mathcal{A}$ correctly guesses the test bit. The advantage is

$$\text{Adv}_{\text{ADKEX}}^{\text{wPFS}}(\mathcal{A}) = \left| \Pr[\text{Succ}_0] - \frac{1}{2} \right|.$$

In the test session, the transcript is $(m_1 = M_1 || ct_s, m_2)$. The real DKEM values are

$$(ss_s, ct_s) \leftarrow \text{DKEM.Encaps}(pk_B)$$

and

$$(ss_e, m_2) \leftarrow \text{DKEM.Encaps}(M_1).$$

The real session key is, except for a correctness error,

$$ss = \text{KDF}(ss_e, ss_s, T),$$

where $T = H(\text{LABEL} || pk_B || m_1 || m_2)$. In the wPFS case, the adversary may learn sk_B , and hence may compute

$$ss_s = \text{DKEM.Decaps}(sk_B, ct_s).$$

Therefore, secrecy cannot rely on ss_s . It must rely on the second DKEM key ss_e , which is encapsulated to the initiator's ephemeral public key M_1 , whose corresponding secret key sk_e is not revealed by freshness.

Game 0': Define game $G_{0'}$ to be game G_0 , except that it aborts if the decapsulations fail in the test session. Let $\text{Succ}_{0'}$ be the event that $\mathcal{A}$ wins in $G_{0'}$.

Therefore,

$$|\Pr[\text{Succ}_0] - \Pr[\text{Succ}_{0'}]| \leq 2\delta.$$

Game 1: Game G_1 is identical to $G_{0'}$, except that in the test session the DKEM key ss_e corresponding to m_2 is replaced, in the session-key computation of both matching local sessions, by an independent random key $ss_r \leftarrow \mathcal{K}$. Let Succ_1 be the event that $\mathcal{A}$ wins in G_1 .

We now show that distinguishing $G_{0'}$ from G_1 implies an IND-CCA attack against the DKEM. Suppose $\mathcal{B}^*$ knows in advance which completed matching session will be tested. The IND-CCA challenger gives $\mathcal{B}^*$

$$(M_1^*, m_2^*, ss_b^*),$$

where $b \leftarrow \{0, 1\}$, $(ss_0^*, m_2^*) = \text{DKEM.Encaps}(M_1^*)$ is the real DKEM key encapsulation under M_1^* , and $ss_1^* \leftarrow \mathcal{K}$ is uniform. The reduction also receives a decapsulation oracle for all ciphertexts $m_2' \neq m_2^*$.

In the known test session, $\mathcal{B}^*$ sets the initiator's ephemeral public key to $M_1 := M_1^*$. It

generates the responder's static key pair honestly:

$$(pk_B, sk_B) \leftarrow \text{DKEM.KeyGen}(),$$

and computes

$$(ss_S, ct_S) \leftarrow \text{DKEM.Encaps}(pk_B).$$

Therefore, the first protocol message is $m_1^* = (M_1^*, ct_S)$. For the responder's reply, $\mathcal{B}^*$ uses the IND-CCA challenge ciphertext $m_2 := m_2^*$ and uses $ss_e := ss_b^*$ as the DKEM key in the test-session key computation. Therefore, the simulated test session key, is

$$ss_b = \text{KDF}(ss_b^* || ss_S || T^*), \text{ where } T^* = H(\text{LABEL} || pk_B || m_1^* || m_2^*).$$

If $b = 0$, then ss_b^* is the real encapsulated key for m_2^* , so the simulation is distributed exactly as G_0 . If $b = 1$, then ss_b^* is uniform and independent, so the simulation is distributed exactly as G_1 .

Simulation of other queries. All non-test sessions are simulated honestly. If $\mathcal{A}$ corrupts the party B , then the static key sk_B is known.

If a decapsulation under the challenge public key M_1^* is needed for some ciphertext $m_2' \neq m_2^*$, then $\mathcal{B}^*$ queries its IND-CCA decapsulation oracle. It never queries m_2^* , because the challenge key ss_b^* is already provided by the IND-CCA challenger. Freshness forbids revealing the test session key or the test session state, so the reduction is never required to output the unknown ephemeral secret key corresponding to M_1^* .

The reduction may not know the ephemeral secret key corresponding to M_1^* . In the test session, it only needs to maintain the accepted key value that would be returned by Test, and freshness forbids Reveal of the test session and its matching session. Therefore, the reduction can consistently assign the common test session component to be ss_b^* in both matching local sessions. When $b = 0$, this is the real DKEM key. When $b = 1$, this is exactly the game hop that replaces the DKEM key by a random key.

Let

$$p_0 = \Pr[\text{Succ}_0], \quad p_1 = \Pr[\text{Succ}_1].$$

The reduction $\mathcal{B}^*$ samples the AKE Test bit internally, so it knows whether $\mathcal{A}$'s final answer is correct. It outputs 0 if $\mathcal{A}$ wins its AKE experiment, and outputs 1 otherwise.

When the IND-CCA bit is $b = 0$, the AKE adversary is playing G_0 . Therefore,

$$\Pr[\mathcal{B}^* \text{ outputs } 0 \mid b = 0] = p_0.$$

When the IND-CCA bit is $b = 1$, the AKE adversary is playing G_1 . Therefore,

$$\Pr[\mathcal{B}^* \text{ outputs } 1 \mid b = 1] = 1 - p_1.$$

Hence,

$$\begin{aligned} \Pr[\mathcal{B}^* \text{ wins}] &= \frac{1}{2} \Pr[\mathcal{B}^* \text{ outputs } 0 \mid b = 0] + \frac{1}{2} \Pr[\mathcal{B}^* \text{ outputs } 1 \mid b = 1] \\ &= \frac{1}{2} p_0 + \frac{1}{2} (1 - p_1) \\ &= \frac{1}{2} + \frac{p_0 - p_1}{2}. \end{aligned}$$

Thus,

$$\text{Adv}_{\text{DKEM}}^{\text{ind-cca}}(\mathcal{B}^*) = \left| \Pr[\mathcal{B}^* \text{ wins}] - \frac{1}{2} \right| = \frac{|p_0 - p_1|}{2}.$$

Equivalently,

$$|p_0 - p_1| \leq 2 \cdot \text{Adv}_{\text{DKEM}}^{\text{ind-cca}}(\mathcal{B}^*).$$

Now consider the case, where the real reduction $\mathcal{B}$ does not know in advance which session $\mathcal{A}$ will test. It therefore guesses one of the at most q_s candidate matching sessions. Therefore,

$$\text{Adv}_{\text{DKEM}}^{\text{ind-cca}}(\mathcal{B}) \geq \frac{1}{q_s} \text{Adv}_{\text{DKEM}}^{\text{ind-cca}}(\mathcal{B}^*).$$

Combining this with the previous inequality, we get:

$$|p_0 - p_1| \leq 2q_s \cdot \text{Adv}_{\text{DKEM}}^{\text{ind-cca}}(\mathcal{B}),$$

and hence,

$$|\Pr[\text{Succ}_0] - \Pr[\text{Succ}_1]| \leq 2q_s \cdot \text{Adv}_{\text{DKEM}}^{\text{ind-cca}}(\mathcal{B}).$$

Game 2: In G_1 , the test-session key is

$$ss = \text{KDF}(ss_r, ss_s, T),$$

where $ss_r \leftarrow \mathcal{K}$. The value ss_s is be known to the adversary after corruption of B , because

$$ss_s = \text{DKEM.Decaps}(sk_B, ct_S).$$

Define G_2 as G_1 , except that the test-session key is replaced by a uniform $K \leftarrow \{0, 1\}^n$.

Let Succ_2 be the event that $\mathcal{A}$ wins in G_2 . We have,

$$\Pr[\text{Succ}_2] = \frac{1}{2}.$$

The only way $\mathcal{A}$ to distinguish $\text{KDF}(ss_r, ss_s, T)$ from a uniform value is to query the KDF at exactly (ss_r, ss_s, T) . Let col be the event that $\mathcal{A}$ makes this query. Therefore, over q_H queries, we have

$$\Pr[\text{col}] \leq \frac{q_H}{2^n}.$$

If col does not occur, then $\text{KDF}(ss_r, ss_s, T)$ is a random value. We have,

$$|\Pr[\text{Succ}_1] - \Pr[\text{Succ}_2]| \leq \Pr[\text{col}] \leq \frac{q_H}{2^n}.$$

Now using the definition

$$\text{Adv}_{\text{ADKEX}}^{\text{wPFS}}(\mathcal{A}) = \left| \Pr[\text{Succ}_0] - \frac{1}{2} \right| = |\Pr[\text{Succ}_0] - \Pr[\text{Succ}_2]|$$

and the triangle inequality, we obtain

$$\text{Adv}_{\text{ADKEX}}^{\text{wPFS}}(\mathcal{A}) \leq 2q_s \cdot \text{Adv}_{\text{DKEM}}^{\text{ind-cca}}(\mathcal{B}) + \frac{q_H}{2^n} + 2\delta.$$

□

5.2 Analysis with respect to known attacks

5.2.1 Attacks against the underlying MLWE problem

The best known attacks against the underlying MLWE problem in ADKEX do not significantly exploit the algebraic structure of module lattices. Several works have investigated whether the additional algebraic structure present in Ring-LWE and Module-LWE instances can be leveraged to improve attacks. More concretely, the works in [28, 29] propose quantum algorithms for recovering short generators of principal ideals in cyclotomic rings, which translates to efficient algorithms for Ideal-SVP in certain settings. Similarly, [30] gave efficient quantum algorithms for the principal ideal problem in arbitrary degree number fields. However, subsequent analysis established that these results do not extend to the module lattice setting in any practically threatening way. In particular, as noted in [19], any attempt to use the reduction in [31] from MLWE to Ring-LWE to attack ADKEX would produce an Ring-LWE instance with a substantially larger modulus and error, making the resulting problem harder to attack than the original MLWE instance. Most recently, theoretical work on module-BKZ has shown that exploiting module structure can yield subexponential speedups for certain cyclotomic fields [32, 33]; however, this speedup applies only to non-power-of-two cyclotomic fields, so the ring used in ADKEX is explicitly outside the scope of this result. While all these algebraic techniques lead to a modest reduction in the estimated cost of certain attacks, the improvement is limited and does not currently lead to practical attacks against standard parameter sets [34]. We therefore analyze the hardness of the MLWE problem as an LWE problem.

Attacks against the LWE problem. Many algorithms exist for solving the LWE problem [35]. However, most combinatorial and algebraic approaches require significantly more samples than are available in the present scheme, which provides only $m = (k + 1)n$ LWE samples. In particular, BKW-type attacks typically need thousands of samples to succeed [36] and are therefore effectively ruled out in this limited-sample regime. Similarly, algebraic attacks such as the Arora–Ge linearization method and Gröbner basis techniques require substantially more samples than available here and are therefore infeasible [37]. Consequently, the relevant attacks are lattice-reduction–based methods using BKZ [38, 39], namely the primal and dual attacks (and later hybrid attacks), which form the primary basis for estimating the hardness of our LWE problem.

The BKZ algorithm works by iteratively calling an oracle that solves the SVP in a reduced sublattice of block size b , and uses the result to improve the basis of the parent lattice. The number of calls to the SVP oracle is known to remain polynomial [40]. As a first simple approach we introduce the core-SVP hardness methodology, which ignores this polynomial factor and considers only the cost of a single SVP oracle call in dimension b [41]. The results of this analysis can be seen in Table 8. While this approach provides useful intuition, it is too coarse to yield accurate security estimates in the classical framework [19]. More precise concrete estimates are therefore provided in Section 5.2.2.

Enumeration vs sieving. Two algorithmic approaches are used for the SVP oracle in BKZ: enumeration and lattice sieving. Enumeration algorithms systematically explore a bounded search tree of candidate vectors in the lattice basis in order to find a shortest vector. They require relatively little memory, but have running time that grows super-exponentially in the block size b . They are thus only useful for small block dimensions. Lattice sieving algorithms instead maintain large lists of lattice vectors and iteratively combine them to produce shorter vectors. They have exponential complexity both in memory and time, and outperform enumeration around dimension 70 [42]. Modern sieving techniques based on locality-sensitive hashing and filtering achieve a heuristic classical time complexity of approximately $2^{0.292b+o(b)}$ [43]. This last value is the one we considered for the core-SVP hardness estimations. Quantum algorithms can further accelerate lattice sieving. Applying Grover-type search techniques reduces the complexity to approximately $2^{0.265b+o(b)}$ [44, 45], and more recent work using quantum hypercone locality-sensitive filtering improves this bound to roughly $2^{0.257b+o(b)}$ [46, 47]. For the core-SVP hardness estimations we adopted this latter value, corresponding to a stronger quantum attacker assumption and therefore a more conservative bound on our quantum security.

Primal and dual lattice attacks. Following the core-SVP methodology, the hardness of the LWE instance is estimated by determining the block size b for which BKZ reduction is expected to succeed. Two main lattice attacks are considered in this framework: the *primal attack* and the *dual attack*.

Primal attack. The primal attack embeds the LWE instance into a lattice in which the vector containing the secret and error terms appears as a uniquely short vector. Given an LWE instance $(\mathbf{A}, \mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e})$, one constructs the lattice $\Lambda = \{\mathbf{x} \in \mathbb{Z}^{m+kn+1} : (\mathbf{A} \mid \mathbf{I}_m \mid -\mathbf{b})\mathbf{x} \equiv \mathbf{0} \pmod{q}\}$ which has dimension $d = m + kn + 1$ and volume q^m . This lattice contains the vector $\mathbf{v} = (\mathbf{s}, \mathbf{e}, 1)$ whose norm is approximately $\|\mathbf{v}\| \approx \sigma\sqrt{kn+m}$, where σ denotes the standard deviation of the secret and error coefficients. The number of used samples m is numerically optimized among the choices from 0 to $(k+1)n$, together with the BKZ block size. The success of BKZ is modeled using the geometric series assumption (GSA), producing a basis whose Gram–Schmidt norms follow approximately $\|\mathbf{b}_i^*\| = \delta^{d-2i-1} \cdot \text{Vol}(\Lambda)^{1/d}$, where $\delta = \left((\pi b)^{1/b} \cdot b/2\pi e\right)^{1/(2(b-1))}$. The short vector $\mathbf{v}$ is expected to be detected by BKZ once its projection onto the subspace spanned by the last b Gram–Schmidt vectors becomes shorter than $\|\mathbf{b}_{d-b}^*\|$, that is, the attack is successful if and only if $\sigma\sqrt{b} \leq \delta^{2b-d-1} \cdot q^{m/d}$ [45, 41, 19]. This corresponds to the unique SVP (uSVP) formulation. Nevertheless, in addition to that, the primal attack can also be viewed as a bounded distance decoding (BDD) problem. After lattice reduction, the target vector lies close enough to the lattice and can be recovered using enumeration if it is within the decoding radius. In practice, both attack variants are considered, and the minimum resulting cost is used as the primal attack cost estimate, which for our parameters is generally the BDD.

Dual attack. The dual attack aims to distinguish LWE samples from uniform by exploiting a statistical bias, so its goal is to find a short vector $\mathbf{w} = (\mathbf{x}, \mathbf{y})$ of length ℓ in the dual lattice $\Lambda^* = \{(\mathbf{x}, \mathbf{y}) \in \mathbb{Z}^m \times \mathbb{Z}^{kn} : \mathbf{A}^T \mathbf{x} \equiv \mathbf{y} \pmod{q}\}$, which has dimension $d = m + kn$ and volume q^{kn} . In this case, $z = \mathbf{x}^T \mathbf{b} = \mathbf{y}^T \mathbf{s} + \mathbf{x}^T \mathbf{e} \pmod{q}$ follows a Gaussian distribution of standard deviation $\ell\sigma$ if $\mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e}$ is an LWE sample and is uniform modulo q otherwise. The maximal statistical distance between these two distributions is bounded by $\varepsilon = 4\exp(-2\pi^2\tau^2)$, where $\tau = \ell\sigma/q$, so a vector of length ℓ yields a distinguishing advantage ε against decision-LWE. Under GSA assumption $\ell \approx \|\mathbf{b}_0\| = \delta^{d-1}q^{kn/d}$, for a vector produced by BKZ. Therefore, obtaining an ε -distinguisher requires running BKZ with block size b such that $-2\pi^2\tau^2 \geq \ln(\varepsilon/4)$. To obtain a constant success probability, the advantage must be amplified by repeating the procedure approximately $1/\varepsilon^2$ times. Since sieving algorithms are expected to output about $2^{0.2075b}$ sufficiently short vectors, the total repetition cost is estimated as $R = \max\left(1, \frac{1}{2^{0.2075b}\varepsilon^2}\right)$. This estimate assumes conservatively for the defender that all vectors returned by the sieving algorithm are as short as the shortest one [41, 19].

Hybrid attacks. Both the primal and the dual attacks have hybrid variants, which combine lattice reduction techniques with an exhaustive search over a subset of the secret. In a hybrid attack, the secret vector is decomposed into two parts, one of which is guessed, while the remaining part is recovered via a reduced lattice problem of lower dimension. This trade-off allows the attacker to balance the combinatorial cost of guessing against the cost of lattice

reduction. For the primal setting, hybrid variants do not typically yield a significant improvement over standard primal attacks, since the structure of the corresponding uSVP instance does not benefit as strongly from partial secret guessing. In contrast, hybrid dual attacks can provide a noticeable improvement over pure dual attacks when the secret is small, as is our case. The best known attacks against such schemes are therefore hybrid dual attacks, which optimize over the number of guessed secret components and then apply a dual lattice reduction to the resulting reduced instance [48, 49, 50].

Parameter set	Attack	Lattice dimension d	Mode	BKZ blocksize β	$\log_2(\text{gates})$
ADKEX-128	Primal	1014	Classical	414	120.9
			Quantum	414	106.4
	Dual	1024	Classical	424	123.8
			Quantum	424	109.0
	Dual hybrid	1024	Classical	395	115.3
			Quantum	401	103.1
ADKEX-256	Primal	1930	Classical	890	259.9
			Quantum	890	228.7
	Dual	1942	Classical	904	264.0
			Quantum	904	232.3
	Dual hybrid	1942	Classical	826	241.2
			Quantum	838	215.4
ADKEX-512	Primal	3686	Classical	1881	549.3
			Quantum	1881	483.4
	Dual	3834	Classical	1890	551.9
			Quantum	1890	485.7
	Dual hybrid	3784	Classical	1732	505.7
			Quantum	1753	450.5

Table 8: Core-SVP hardness of the scheme (using ADPS16 [35] and ChaLoy21 [47] models for classical and quantum estimates respectively in the Lattice Estimator [51, 26]).

5.2.2 Refined gate-count estimate

The core-SVP hardness methodology is too coarse to produce accurate concrete security estimates in the classical setting. To obtain refined estimates, we rely on the Lattice Estimator of Albrecht et al. [51], which is the most widely accepted tool in the cryptographic community for this purpose.

The estimator models lattice reduction using progressive BKZ, in which the BKZ block size increases gradually throughout the reduction process rather than being fixed from the start. This is generally considered more realistic than fixed-block BKZ analyses. We configure the estimator to use the Gama–Nguyen basis shape simulator (GN11) [52] in place of GSA to model the Gram–Schmidt profile of the reduced basis. Unlike GSA, GN11 is calibrated from experimental BKZ data and captures tail effects in the Gram–Schmidt norms that GSA systematically misses.

This provides more accurate predictions of BKZ behavior at the large block sizes relevant to cryptographic security estimates.

For the sieving cost model, we use the MATZOV reduction cost model [53] as implemented in the estimator. The estimator automatically accounts for all BKZ reduction costs, progressive overheads, and dimension reductions (dimensions-for-free [54]). No additional multiplicative constants or overhead factors are applied manually, unlike earlier analyses such as in [19].

In the original Kyber round-3 security analysis [19], the primal attack was identified as the dominant attack, with the dual attack providing a secondary reference point at comparable or slightly higher cost. Since then, the landscape of dual attacks has evolved significantly. Guo and Johansson [55] and, independently, the MATZOV team [53] proposed substantial improvements to the dual attack by incorporating a Fast Fourier Transform (FFT) step into the dual-sieve framework, claiming reductions of 5–15 bits in the concrete security of Kyber parameter sets, with potentially similar impact on comparable parameter sets. These results attracted significant attention as they appeared to threaten the security margins of several NIST candidates. However, Ducas and Pulles [56] showed that a key *independence heuristic* underlying these improved dual attacks leads to incorrect predictions in parameter regimes directly relevant to cryptographic applications. Their analysis suggests that the primal attack likely regains the status of the most efficient classical attack under a corrected model. Carrier et al. subsequently proposed a rigorous variant of the MATZOV dual attack in [57], replacing modulus switching with an efficient decoding algorithm based on polar codes over $\mathbb{Z}_q$, and confirmed the claimed improvements under this corrected framework, partially restoring the MATZOV security reductions.

Given this landscape, we adopt the MATZOV reduction cost model [53] as a conservative lower bound on the security of ADKEX. We emphasize that the independence heuristic underlying the MATZOV dual attack has been called into question in [56], but the corrected analysis in [57] yields results of comparable magnitude under a rigorous foundation. The primal attack costs are not affected by this controversy at all.

Parameter Set	Attack	Lattice dimension d	BKZ blocksize β	$\log_2(\text{gates})$
ADKEX-128	Primal	1009	396	142.1
	Dual	1024	424	149.9
	Dual hybrid	1024	387	139.7
ADKEX-256	Primal	1914	870	275.4
	Dual	1954	918	288.5
	Dual hybrid	1954	823	262.3
ADKEX-512	Primal	3674	1855	552.9
	Dual	3866	1930	574.0
	Dual hybrid	3836	1736	519.9

Table 9: Refined classical security estimates using the MATZOV reduction cost model [53] and GN11 basis shape simulator [52], as implemented in the Lattice Estimator [51, 26].

The resulting estimates are summarized in Table 9. They represent a lower bound on the true

classical security level: the true cost to an adversary is at least as large as reported, and likely larger under more optimistic models for the defender. As shown, the most cost-efficient attack across all parameter sets is the dual-hybrid attack. In this case, the proposed parameter sets achieve classical security levels of 139.7, 262.3, and 519.9 bits for ADKEX-128, ADKEX-256, and ADKEX-512 respectively. These margins comfortably exceed the target security levels of 128, 256, and 512 bits [27]. Under less aggressive attack models the true security margins are expected to be higher.

Regarding lattice reduction costs, the most demanding case (ADKEX-128) requires BKZ with block size $\beta = 387$. For comparison, the current record for solving SVP instances as of January 2026 is dimension 210 [58], which remains significantly below the block sizes required to attack any ADKEX parameter set.

All reported costs are heuristic and rely on assumptions about lattice reduction behavior, SVP oracle complexity, and hybrid attack efficiency. The resulting estimates differ from earlier analyses such as [19], which did not account for hybrid attacks, relied on less refined cost models, and did not incorporate the developments in dual attack methodology described above.

5.2.3 About quantum security

Estimating the concrete security of lattice-based constructions against quantum adversaries remains an open problem, and no community consensus currently exists on a definitive refined cost model analogous to those available for classical attacks. We therefore adopt a core-SVP-based approach, following standard practice in analyses of schemes such as ML-KEM [19], modeling the cost of the quantum SVP oracle as 2^{cb} for block size b . A natural choice is $c = 0.265$, corresponding to the cost of Grover-accelerated lattice sieving [44, 45]. In this work, however, we adopt the slightly tighter value $c = 0.257$ from Chailloux and Loyer [47], which uses quantum random walks instead of plain Grover search and represents a more conservative (attacker-favorable) security assumption. In this setting, the quantum security bits are 103.1, 215.4 and 450.5 for ADKEX-128, ADKEX-256 and ADKEX-512 respectively (see Table 8). These values significantly exceed the target levels of 80, 128, and 256 quantum security bits [27].

5.2.4 Attacks against symmetric primitives

The symmetric primitives currently used in the reference implementation of ADKEX are SM3-based constructions provided by the NGCC auxiliary function library, specifically `sm3hash`, `pseudoX0F`, and `pseudohash`.

The NGCC documentation explicitly states that `pseudoX0F` and `pseudohash` are provided solely for correctness verification in the current evaluation round and do not guarantee security, and that they will be replaced by standardized primitives in subsequent rounds [27]. The security analysis of ADKEX with respect to its symmetric components is therefore conditional: we

assume that the final ICCS-standardized replacements will satisfy the properties required by the security proof, namely pseudorandomness for the XOF and PRF components, and collision resistance for the hash components.

Regarding SM3 itself, no attack on the full construction is currently known. Known crypt-analytic results are limited to step-reduced variants and do not threaten the full 64-round function [59, 60]. The security proofs model the hash and XOF functions as random oracles. Should the ICCS replacement primitives be based on a different underlying function, this analysis will need to be updated accordingly.

5.2.5 Attacks exploiting reconciliation failures

Reconciliation failure probability δ plays a role in the attacker’s advantage. A first naive approach for an attacker would be to try random ciphertexts. This has a success probability of $q_d \delta$, where q_d is the number of decapsulation queries. With up to 2^{80} ciphertext queries (NGCC adversary model [27]), the probability of triggering at least one failure is at most $2^{-51.7}$, $2^{-100.2}$ and $2^{-86.0}$ for ADKEX-128, ADKEX-256 and ADKEX-512 respectively, which is negligible.

A more sophisticated adversary wishing to exploit failures proceeds as follows: rather than submitting random ciphertexts, the adversary searches offline for values of coins (Line 1 of Alg. 15) such that the derived randomness r produces response-side noise vectors with unusually large norm. In fact, larger response-side noise increases the magnitude of the inner products involved in reconciliation, raising the chance that the reconciliation threshold is exceeded and a failure occurs. This strategy is known as *failure boosting* [61] and is especially useful when the adversary is constrained to the number of decapsulation queries (2^{80} in the NGCC model [27]), or has access to a quantum computer (since Grover’s algorithm can speed up this offline search quadratically). However, this search faces two compounding obstacles. First, since XOF behaves as a random oracle, there is no shortcut: finding coins that produce large-norm noise requires an exhaustive search, and high-dimensional noise vectors are extremely unlikely to deviate significantly from their expected size, so the search remains exponentially expensive even with Grover’s quadratic speedup. Second, even if the adversary finds larger noise, the actual failure also depends on the initiator’s secret key noise, which is unknown, so the improvement in failure probability per query is only modest. The gain from finding large noise is therefore far outweighed by the cost of finding it, for all our parameter sets [61, 62, 19].

From one to multiple failures. As explained in [19], an attacker might need to trigger more than one failure to mount a key recovery attack. The security of the scheme, however, must not depend on the difficulty of triggering many failures, since the work in [63] showed that once an adversary finds a single failing ciphertext, finding further ones becomes dramatically cheaper, because the first failure reveals information about the direction of the secret key noise, which can be exploited to craft ciphertexts that are much more likely to fail. The implication is that the

security thus relies on whether the adversary can trigger even one failure. Our parameter sets are chosen precisely so that this is computationally infeasible.

Multitarget attacks. An adversary targeting many users simultaneously might precompute some failure-inducing coins and attempt to reuse them across many victims [64]. Nevertheless, in line 2 of Alg. 14 this kind of attack is prevented, since ρ (the last n bits of the encapsulation key) is hashed into the randomness. A different user’s key embeds a different ρ , so the same coins value produces a completely independent r . This is the direct analogue of ML-KEM’s protection via hashing the public key into the encapsulation randomness [19].

5.3 Other security properties

Forward secrecy. ADKEX provides forward secrecy through the use of fresh ephemeral key material in every protocol execution. For each session, the initiator generates a new ephemeral public and secret key for each session. This key is used exclusively within the current session and is discarded upon protocol completion. Thus, since the ephemeral secret is independently generated for each protocol invocation and is never reused across sessions, each session key remains cryptographically independent of past and future sessions.

Side-channel resistance. ADKEX is designed for constant-time implementation on all target platforms. The performance-critical, secret-dependent operations (NTT-based polynomial arithmetic, centered-binomial sampling, modular reduction, compression and serialization) are inherited directly from the constant-time, public-domain ML-KEM code bases (PQClean [65], `mlkem-native` [66]) and from the constant-time Plantard arithmetic of [67], none of which use secret-dependent branches, memory indices, or variable-time divisions. The components specific to ADKEX are implemented with the same discipline in all our implementations. The two-layer structure delegates all long-term-secret-dependent computation to the `DeriveSecret` step, which simplifies side-channel review and localizes the future addition of masking.

Symmetric-layer side channels. Because the symmetric layer is built entirely on SM3, ADKEX can draw directly on the hardware and constant-time SM3 implementations available in the commercial cryptography ecosystem, reducing the side-channel exposure of hashing, key derivation, and pseudorandom expansion on deployed platforms.

6 Feature Statements

This section provides explicit feature statements for DKEM against each of the criteria defined by the NGCC [27]: innovativeness, simplicity, flexibility, compatibility, extensibility, and

performance advantages on variant platforms. Each feature is argued from specific verifiable design decisions documented in the preceding sections.

Innovativeness The main innovations of ADKEX are on the used key encapsulation mechanism, DKEM. The central innovation of DKEM is its construction lineage. Every MLWE KEM in current international standards builds a shared secret using the LPR framework: the encapsulator embeds a random message in a noisy ciphertext, the decapsulator recovers the message via approximate decryption, and the session key is derived. DKEM does not encrypt a message: both parties instead compute approximations to the same lattice product from their respective MLWE samples and use a signal function to extract one bit of shared secret per coefficient, adapting the approach in [5, 6] to MLWE.

The second major innovation is the tag based Fujisaki-Okamoto (FO) transform. As no message is encrypted, DKEM’s core has no plaintext to recover, so the standard FO template does not apply [68]. In order to achieve CCA security, the tag-based FO transform addresses this by treating the encapsulation randomness as the transported artifact, masking it with the CPA shared secret in a tag appended to the ciphertext. The decapsulator recovers the randomness from the tag and re-runs encapsulation to verify ciphertext integrity.

Moreover, the reconciliation mechanism of [5, 6] is adapted to the module setting and tuned to the ML-KEM ring. Transmitting ℓ bits per coefficient over 2^ℓ signal subintervals to trade a small amount of bandwidth for a markedly lower failure probability is the general *key-consensus* approach studied in [20, 21] (and, for $\ell = 1$, the reconciliation of [5, 6, 41]). The contribution of ADKEX here is the instantiation of this mechanism on the standardized ML-KEM parameters, together with a self-contained proof (Lemma 2.1) that the extracted bit is perfectly biased given the signal under our randomized-doubling step.

ADKEX does not introduce a new key-exchange construction since it follows the KEMTLS-PDK pattern [10], instantiated with DKEM in both the static and ephemeral KEM roles. Relative to KEMTLS-PDK it is deliberately minimal, simplifying only the two key-establishment flights, replacing the staged TLS 1.3 key schedule with a single transcript-bound derivation $ss = \text{KDF}(ss_e, |, ss_s, |, T)$. Its specific contribution is one of profile rather than structure: authentication is carried entirely by the KEM, making ADKEX a signature-free authenticated key exchange built from a single primitive (in contrast to the signature-based sibling **DKEX**), and with DKEM’s SM3-only symmetric layer every component — KEM, hash, XOF, and KDF — is a domestic SM primitive.

Simplicity. ADKEX achieves authenticated key establishment in two message flows without any signature operations. A single primitive, DKEM, serves both the ephemeral and the static roles, so an implementation requires no additional cryptographic dependencies beyond those already present in DKEM.

Also, with regard to DKEM, it uses a single noise parameter η across all secret and error

vectors at every security level, in contrast to ML-KEM’s two-parameter design. This unifies the sampling into a single CBD_η instantiation, reduces code footprint and hardware area, and eliminates a class of implementation errors arising from applying the wrong parameter in the wrong context.

Flexibility. ADKEX inherits the three parameter sets of DKEM directly, covering classical security strengths of at least 128, 256, and 512 bits with corresponding quantum-resistant security of at least 80, 128, and 256 bits, spanning the full range mandated by the Submission Requirements [27]. All security levels are covered by the same two-flow protocol structure with no structural changes.

As requested by NGCC, *algorithms can be implemented securely and efficiently on a wide variety of platforms, supporting various tradeoffs between efficiency and cost.* Concretely, in contrast to more costly Gaussian sampling, ADKEX samples secret and noise polynomials using CBD, which can be implemented securely and efficiently across various platforms [41]. Besides, as demonstrated by existing ML-KEM/ML-DSA optimizations, the underlying polynomial arithmetic in ADKEX can likewise be efficiently adapted to a broad range of platforms, from high-performance platforms [69, 70, 71] to resource-constrained devices [67, 72, 73]. Furthermore, ADKEX can support various time-memory trade-offs on highly constrained IoT devices [74, 75, 76], providing substantial flexibility for secure and efficient deployment across diverse platforms. All three security levels are supported from a shared codebase with compile-time selection, minimizing deployment and maintenance burden.

Compatibility. The symmetric layer of ADKEX is inherited directly from DKEM (hash function, XOF, KDF, and rejection hash), which is entirely based on SM3, making the scheme natively compatible with the Chinese SM cryptographic suite at the specification level.

Furthermore, being based on the MLWE problem, ADKEX shares the same mathematical foundation as the dominant PQC standardized schemes. Moreover, for the 128 and 256-bit parameter sets, it uses the same ring, modulus and rank as ML-KEM-512 and ML-KEM-1024, inheriting also the NTT structure. All this allows existing ML-KEM implementations to contribute, and enables existing research and cryptanalysis exposure to apply to this scheme.

Extensibility. ADKEX’s core is the key encapsulation mechanism **DKEM**, which is on its own a CCA secure KEM, and is submitted as separate submissions in this ICCS call. Moreover, its CPA core, DKE, is a complete two-party key exchange protocol, where both parties derive the same n -bit shared secret. Thus, inside this same specification there is a whole complete CPA-secure key exchange protocol that could be used for any case where CCA security is not a requirement. On the other hand, a structureless-LWE variant can also be obtained by replacing the module polynomial arithmetic with plain-LWE matrix operations while retaining

the signal function, mod-2 extractor, and tag-based FO transform unchanged, providing an assumption-diversity extension consistent with the NGCC emphasis on hard-problem diversity.

Performance Advantages on Variant Platforms. The reference implementation executes the complete two-pass ADKEX exchange using portable C, while the performance optimization version preserves the protocol logic and communication format and uses the x86-64 AVX2 DKEM backend for the performance-critical polynomial-arithmetic operations. The complete-exchange measurements reported in the Performance Evaluation section show mean execution-time reductions of approximately 42.4%, 29.7%, and 23.6% for ADKEX-128, ADKEX-256, and ADKEX-512, respectively, without changing the transmitted or stored data sizes. This separation allows the DKEM arithmetic backend to be optimized for a specific instruction set without modifying the authenticated key-exchange layer.

For resource-constrained platforms, ADKEX also includes a Cortex-M4 implementation using optimized Plantard arithmetic and an assembly-optimized SM3 backend. The additional QEMU evidence covers all three parameter sets and reports stage cycle counts for initialization, both protocol passes, and shared-secret derivation. These implementations provide evidence of instruction-set acceleration on x86-64 and functional execution on a Cortex-M4 target.

References

- [1] Peter W. Shor. Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer. *SIAM Journal on Computing*, 26(5):1484–1509, 1997.
- [2] Oded Regev. On Lattices, Learning with Errors, Random Linear Codes, and Cryptography. In *Proceedings of the Thirty-Seventh Annual ACM Symposium on Theory of Computing (STOC)*, pages 84–93. ACM, 2005.
- [3] Vadim Lyubashevsky, Chris Peikert, and Oded Regev. On Ideal Lattices and Learning with Errors over Rings. In *Advances in Cryptology – EUROCRYPT 2010*, volume 6110 of *Lecture Notes in Computer Science*, pages 1–23. Springer, 2010.
- [4] Adeline Langlois and Damien Stehlé. Worst-Case to Average-Case Reductions for Module Lattices. *Designs, Codes and Cryptography*, 75:565–599, 2015.
- [5] Jintai Ding, Xiang Xie, and Xiaodong Lin. A Simple Provably Secure Key Exchange Scheme Based on the Learning with Errors Problem. Cryptology ePrint Archive, Paper 2012/688, 2012.
- [6] Jintai Ding, Tsuyoshi Takagi, Xinwei Gao, and Yuntao Wang. Ding Key Exchange. NIST’s call for Post-quantum Cryptography Standardization, 2017.
- [7] Joppe Bos, Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, John M. Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehlé. CRYSTALS – Kyber: a CCA-secure module-lattice-based KEM. Cryptology ePrint Archive, Paper 2017/634, 2017.
- [8] National Institute of Standards and Technology. FIPS 203: ML-KEM: Module Lattice-Based Key-Encapsulation Mechanism. <https://doi.org/10.6028/NIST.FIPS.203>, 2024. U.S. Department of Commerce, August 2024.
- [9] National Institute of Standards and Technology. FIPS 204: Module-Lattice-Based Digital Signature Standard. Federal Information Processing Standards Publication 204, 2024.
- [10] Peter Schwabe, Douglas Stebila, and Thom Wiggers. More efficient post-quantum KEMTLS with pre-distributed public keys. Cryptology ePrint Archive, Paper 2021/779, 2021.
- [11] Colin Boyd, Bor de Kock, and Lise Millerjord. Modular Design of KEM-Based Authenticated Key Exchange. In *Information Security and Privacy: 28th Australasian Conference, ACISP 2023, Brisbane, QLD, Australia, July 5–7, 2023, Proceedings*, page 553–579, Berlin, Heidelberg, 2023. Springer-Verlag.

- [12] Jiaxin Pan, Benedikt Wagner, and Runzhi Zeng. Lattice-Based Authenticated Key Exchange with Tight Security. In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology – CRYPTO 2023*, pages 616–647, Cham, 2023. Springer Nature Switzerland.
- [13] Ran Canetti and Hugo Krawczyk. Analysis of Key-Exchange Protocols and Their Use for Building Secure Channels. In Birgit Pfitzmann, editor, *Advances in Cryptology — EUROCRYPT 2001*, pages 453–474, Berlin, Heidelberg, 2001. Springer Berlin Heidelberg.
- [14] Atsushi Fujioka, Koutarou Suzuki, Keita Xagawa, and Kazuki Yoneyama. Strongly Secure Authenticated Key Exchange from Factoring, Codes, and Lattices. *Designs, Codes and Cryptography*, 76(3):469–504, September 2015.
- [15] Simon Blake-Wilson and Alfred Menezes. Unknown Key-Share Attacks on the Station-to-Station (STS) Protocol. In *Public Key Cryptography*, pages 154–170, Berlin, Heidelberg, 1999. Springer Berlin Heidelberg.
- [16] Liqun Chen and Qiang Tang. Bilateral Unknown Key-Share Attacks in Key Agreement Protocols. *J. Univers. Comput. Sci.*, 14:416–440, 2008.
- [17] Cas Cremers, Alexander Dax, and Niklas Medinger. Keeping Up with the KEMs: Stronger Security Notions for KEMs and Automated Analysis of KEM-based Protocols. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security, CCS '24*, page 1046–1060, New York, NY, USA, 2024. Association for Computing Machinery.
- [18] Vadim Lyubashevsky and Gregor Seiler. NTTRU: Truly Fast NTRU Using NTT. *Cryptology ePrint Archive*, Paper 2019/040, 2019.
- [19] Roberto Avanzi, Joppe Bos, Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, John M. Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehlé. CRYSTALS-Kyber Algorithm Specifications and Supporting Documentation (Version 3.02). Submission to the NIST Post-Quantum Cryptography Standardization Project, Round 3, 2021. Updated August 4, 2021.
- [20] Zhengzhong Jin and Yunlei Zhao. Optimal Key Consensus in Presence of Noise. *Cryptology ePrint Archive*, Paper 2017/1058, 2017.
- [21] Yunlei Zhao, Zhengzhong Jin, Boru Gong, and Guanye Sui. KCL (pka OKCN/AKCN/CNKE): Key Consensus from Lattice. Submission to NIST PQC Standardization Project, Round 1.
- [22] James W. Cooley and John W. Tukey. An Algorithm for the Machine Calculation of Complex Fourier Series. *Mathematics of Computation*, 19(90):297–301, 1965.

- [23] W. Morven Gentleman and Gordon Sande. Fast Fourier Transforms: For Fun and Profit. In *Proceedings of the November 7–10, 1966, Fall Joint Computer Conference (AFIPS '66)*, pages 563–578, 1966.
- [24] Patrick Longa and Michael Naehrig. Speeding up the Number Theoretic Transform for Faster Ideal Lattice-Based Cryptography. In *Cryptology and Network Security (CANS 2016)*, volume 10052 of *Lecture Notes in Computer Science*, pages 124–139. Springer, 2016.
- [25] Gregor Seiler. Faster AVX2 optimized NTT multiplication for Ring-LWE lattice cryptography. Cryptology ePrint Archive, Paper 2018/039, 2018.
- [26] *Basque Centre for Applied Mathematics*. DKE Scripts: Failure-Probability, Communication-Cost and Concrete-Security Analysis Tools for DKEM. GitHub repository, 2026. Accessed 2026-06-24.
- [27] Institute of Commercial Cryptography Standards. Announcement on Launching the Next-generation Commercial Cryptographic Algorithms Program (NGCC), October 2025.
- [28] Ronald Cramer, Léo Ducas, Chris Peikert, and Oded Regev. Recovering Short Generators of Principal Ideals in Cyclotomic Rings. In *Advances in Cryptology – EUROCRYPT 2016*, volume 9666 of *Lecture Notes in Computer Science*, pages 559–585. Springer, 2016.
- [29] Ronald Cramer, Léo Ducas, and Benjamin Wesolowski. Short Stickelberger Class Relations and Application to Ideal-SVP. In *Advances in Cryptology – EUROCRYPT 2017*, volume 10210 of *Lecture Notes in Computer Science*, pages 324–348. Springer, 2017.
- [30] Jean-François Biasse and Fang Song. Efficient Quantum Algorithms for Computing Class Groups and Solving the Principal Ideal Problem in Arbitrary Degree Number Fields. In *Proceedings of the Twenty-Seventh Annual ACM-SIAM Symposium on Discrete Algorithms (SODA 2016)*, pages 893–902. SIAM, 2016.
- [31] Martin R. Albrecht and Amit Deo. Large Modulus Ring-LWE $\geq$ Module-LWE. Cryptology ePrint Archive, Paper 2017/612, 2017.
- [32] Léo Ducas, Lynn Engelberts, and Paola de Perthuis. Predicting Module-Lattice Reduction. Cryptology ePrint Archive, Paper 2025/1904, 2025.
- [33] Paola de Perthuis and Filip Trenkić. Refined Modelling of the Primal Attack, and Variants Against Module-Learning With Errors. Cryptology ePrint Archive, Paper 2025/2195, 2025.
- [34] Han Wu and Guangwu Xu. Enhancing the Dual Attack against MLWE: Constructing More Short Vectors Using Its Algebraic Structure. Cryptology ePrint Archive, Paper 2022/1661, 2022.

- [35] Martin R. Albrecht, Rachel Player, and Sam Scott. On the concrete hardness of Learning with Errors. Cryptology ePrint Archive, Paper 2015/046, 2015.
- [36] Yu Wei, Lei Bi, Xianhui Lu, and Kunpeng Wang. Memory-Efficient BKW Algorithm for Solving the LWE Problem. Cryptology ePrint Archive, Paper 2025/266, 2025.
- [37] Koen de Boer and Wessel van Woerden. Lattice-based Cryptography: A Survey on the Security of the Lattice-based NIST Finalists. Cryptology ePrint Archive, Paper 2025/304, 2025.
- [38] Claus-Peter Schnorr and Martin Euchner. Lattice Basis Reduction: Improved Practical Algorithms and Solving Subset Sum Problems. *Mathematical Programming*, 66(1–3):181–199, 1994.
- [39] Yuanmi Chen and Phong Q. Nguyen. BKZ 2.0: Better Lattice Security Estimates. In Dong Hoon Lee and Xiaoyun Wang, editors, *Advances in Cryptology – ASIACRYPT 2011*, volume 7073 of *Lecture Notes in Computer Science*, pages 1–20. Springer, 2011.
- [40] Guillaume Hanrot, Xavier Pujol, and Damien Stehlé. Terminating BKZ. Cryptology ePrint Archive, Paper 2011/198, 2011.
- [41] Erdem Alkim, Léo Ducas, Thomas Pöppelmann, and Peter Schwabe. Post-quantum Key Exchange — A New Hope. In *Proceedings of the 25th USENIX Security Symposium (USENIX Security 16)*, pages 327–343, Austin, TX, USA, 2016. USENIX Association.
- [42] Martin R. Albrecht, Léo Ducas, Gottfried Herold, Elena Kirshanova, Eamonn W. Postlethwaite, and Marc Stevens. The General Sieve Kernel and New Records in Lattice Reduction. Cryptology ePrint Archive, Paper 2019/089, 2019.
- [43] Anja Becker, Léo Ducas, Nicolas Gama, and Thijs Laarhoven. New directions in nearest neighbor searching with applications to lattice sieving. Cryptology ePrint Archive, Paper 2015/1128, 2015.
- [44] Thijs Laarhoven. *Search Problems in Cryptography: From Fingerprinting to Lattice Sieving*. PhD thesis, Eindhoven University of Technology, 2016.
- [45] Martin R. Albrecht, Vlad Gheorghiu, Eamonn W. Postlethwaite, and John M. Schanck. Estimating Quantum Speedups for Lattice Sieves. Cryptology ePrint Archive, Paper 2019/1161, 2019.
- [46] Max Heiser. Improved Quantum Hypercone Locality Sensitive Filtering in Lattice Sieving. Cryptology ePrint Archive, Paper 2021/1295, 2021.

- [47] André Chailloux and J. Loyer. Lattice Sieving via Quantum Random Walks. In Mihir Tibouchi and Huaxiong Wang, editors, *Advances in Cryptology – ASIACRYPT 2021 – 27th International Conference on the Theory and Application of Cryptology and Information Security, Singapore, December 6–10, 2021, Proceedings, Part IV*, volume 13093 of *Lecture Notes in Computer Science*, pages 63–91. Springer, 2021.
- [48] Thomas Espitau, Antoine Joux, and Natalia Kharchenko. On a Hybrid Approach to Solve Small Secret LWE, 2020.
- [49] Martin R. Albrecht. On dual lattice attacks against small-secret LWE and parameter choices in HELib and SEAL. Cryptology ePrint Archive, Paper 2017/047, 2017.
- [50] Jung Hee Cheon, Minki Hhan, Seungwan Hong, and Yongha Son. A Hybrid of Dual and Meet-in-the-Middle Attack on Sparse and Ternary Secret LWE. *IACR Cryptology ePrint Archive*, 2019:1114, 2019.
- [51] Martin R. Albrecht et al. Lattice-Estimator. <https://github.com/malb/lattice-estimator>, 2015–. Accessed: 2026-03-24.
- [52] Nicolas Gama and Phong Q. Nguyen. Predicting Lattice Reduction. In Nigel Smart, editor, *Advances in Cryptology – EUROCRYPT 2008*, volume 4965 of *Lecture Notes in Computer Science*, pages 31–51, Berlin, Heidelberg, 2008. Springer.
- [53] MATZOV. Report on the Security of LWE: Improved Dual Lattice Attack. Report, Zenodo, April 2022.
- [54] Léo Ducas. Shortest Vector from Lattice Sieving: A Few Dimensions for Free. In Jesper Buus Nielsen and Vincent Rijmen, editors, *Advances in Cryptology – EUROCRYPT 2018*, volume 10820 of *Lecture Notes in Computer Science*, pages 125–145, Cham, 2018. Springer.
- [55] Qian Guo and Thomas Johansson. Faster Dual Lattice Attacks for Solving LWE with Applications to CRYSTALS. In Mehdi Tibouchi and Huaxiong Wang, editors, *Advances in Cryptology – ASIACRYPT 2021*, volume 13093 of *Lecture Notes in Computer Science*, pages 33–62, Cham, 2021. Springer.
- [56] Léo Ducas and Ludo N. Pulles. Does the Dual-Sieve Attack on Learning with Errors Even Work? In *Advances in Cryptology – CRYPTO 2023*, volume 14083 of *Lecture Notes in Computer Science*, pages 37–69, Cham, 2023. Springer.
- [57] Kévin Carrier, Charles Meyer-Hilfiger, Yixin Shen, and Jean-Pierre Tillich. Assessing the Impact of a Variant of MATZOV’s Dual Attack on Kyber. In *Advances in Cryptology – CRYPTO 2025*, volume 16000 of *Lecture Notes in Computer Science*, pages 444–476, Cham, 2025. Springer.

- [58] TU Darmstadt Lattice Challenge. Svp challenge. <https://www.latticechallenge.org/svp-challenge/>, 2026. Accessed: 2026-03-26.
- [59] Florian Mendel, Tomislav Nad, and Martin Schl  ffer. Finding Collisions for Round-Reduced SM3, 2012.
- [60] Jian Zou, Wenling Wu, Shuang Wu, Bozhan Su, and Le Dong. Preimage Attacks on Step-Reduced SM3 Hash Function. In *Information Security and Cryptology – ICISC 2011*, 2012.
- [61] Jan-Pieter D’Anvers, Frederik Vercauteren, and Ingrid Verbauwhede. On the Impact of Decryption Failures on the Security of LWE/LWR Based Schemes. IACR Cryptology ePrint Archive, Report 2018/1089, 2018.
- [62] Jan-Pieter D’Anvers, Qian Guo, Thomas Johansson, Alexander Nilsson, Frederik Vercauteren, and Ingrid Verbauwhede. Decryption Failure Attacks on IND-CCA Secure Lattice-Based Schemes. In *Public-Key Cryptography – PKC 2019*, volume 11443 of *Lecture Notes in Computer Science*, pages 565–598. Springer, 2019.
- [63] Jan-Pieter D’Anvers, M  lissa Rossi, and Fernando Virdia. (One) Failure Is Not an Option: Bootstrapping the Search for Failures in Lattice-Based Encryption Schemes. In *Advances in Cryptology – EUROCRYPT 2020, Part III*, volume 12107 of *Lecture Notes in Computer Science*, pages 3–33. Springer, 2020.
- [64] Jan-Pieter D’Anvers and Senne Batsleer. Multitarget Decryption Failure Attacks and their Application to Saber and Kyber. IACR Cryptology ePrint Archive, Report 2021/193, 2021.
- [65] Matthias J. Kannwischer, Peter Schwabe, Douglas Stebila, and Thom Wiggers. Improving Software Quality in Cryptography Standardization Projects. In *IEEE European Symposium on Security and Privacy, EuroS&P 2022 - Workshops, Genoa, Italy, June 6-10, 2022*, pages 19–30, Los Alamitos, CA, USA, 2022. IEEE Computer Society.
- [66] Hanno Becker, Matthias J. Kannwischer, and The mlkem-native project authors. mlkem-native: A secure, fast, and portable C90 implementation of ML-KEM / FIPS 203, 2026. Accessed: 2026-06-23.
- [67] Junhao Huang, Jipeng Zhang, Haosong Zhao, Zhe Liu, Ray C. C. Cheung,   etin Kaya Ko  , and Donglong Chen. Improved Plantard arithmetic for lattice-based cryptography. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2022(4):614–636, 2022.
- [68] Dennis Hofheinz, Kathrin H  velmanns, and Eike Kiltz. A Modular Analysis of the Fujisaki-Okamoto Transformation. Cryptology ePrint Archive, Paper 2017/604, 2017.

- [69] Lipeng Wan, Fangyu Zheng, Guang Fan, Rong Wei, Lili Gao, Yuewu Wang, Jingqiang Lin, and Jiankuo Dong. A novel high-performance implementation of CRYSTALS-Kyber with AI accelerator. In *European Symposium on Research in Computer Security*, pages 514–534. Springer, 2022.
- [70] Xinyi Ji, Jiankuo Dong, Tonggui Deng, Pinchang Zhang, Jiafeng Hua, and Fu Xiao. HI-Kyber: A novel high-performance implementation scheme of Kyber based on GPU. *IEEE Transactions on Parallel and Distributed Systems*, 35(6):877–891, 2024.
- [71] Xuan Yu, Jipeng Zhang, Junhao Huang, Donglong Chen, and Lu Zhou. Multi-way High-Throughput Implementation of Kyber. In *International Conference on Information Security*, pages 41–60. Springer, 2024.
- [72] Jipeng Zhang, Yuxing Yan, Junhao Huang, and Çetin Kaya Koç. Optimized software implementation of Keccak, Kyber, and Dilithium on RV {32, 64} IM {B}{V}. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2025(1):632–655, 2025.
- [73] DongHyun Shin, YoungBeom Kim, Ayesha Khalid, Maire O’Neill, and Seog Chung Seo. Optimized Implementations of Keccak, Kyber, and Dilithium on the MSP430 Microcontroller. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2026(2):409–440, 2026.
- [74] Leon Botros, Matthias J Kannwischer, and Peter Schwabe. Memory-efficient high-speed implementation of Kyber on Cortex-M4. In *International Conference on Cryptology in Africa*, pages 209–228. Springer, 2019.
- [75] Denisa O. C. Greconici, Matthias J. Kannwischer, and Amber Sprenkels. Compact Dilithium Implementations on Cortex-M3 and Cortex-M4. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pages 1–24, 2021.
- [76] Joppe W Bos, Joost Renes, and Amber Sprenkels. Dilithium for Memory Constrained Devices. In *International Conference on Cryptology in Africa*, pages 217–235. Springer, 2022.